



CENTRO UNIVERSITÁRIO ALVES FARIA (UNIALFA)
GRADUAÇÃO EM SISTEMAS DE INFORMAÇÃO E ENGENHARIA DE
SOFTWARE

Eduardo Ribeiro Arruda
Carlos Daniel Dias Freitas
João Victor Barbosa Dourado
João Marcello Pereira do Prado

DESENVOLVIMENTO DE UM SISTEMA DE GESTÃO FINANCEIRA
PESSOAL (CORE 4 ERP): UMA ABORDAGEM FOCADA EM
ARQUITETURA DE SOFTWARE, USABILIDADE E INTEGRIDADE DE
DADOS

GOIÂNIA
2026

Eduardo Ribeiro Arruda
Carlos Daniel Dias Freitas
João Victor Barbosa Dourado
João Marcello Pereira do Prado

**DESENVOLVIMENTO DE UM SISTEMA DE GESTÃO FINANCEIRA
PESSOAL (CORE 4 ERP): UMA ABORDAGEM FOCADA EM
ARQUITETURA DE SOFTWARE, USABILIDADE E INTEGRIDADE DE
DADOS**

Trabalho de Conclusão de Curso
apresentado como requisito parcial à
obtenção do grau de Bacharel em Sistemas
de Informação e Engenharia de Software do
Centro Universitário Alves Faria
(UNIALFA), sob orientação docente.

Orientador(a): Prof. Esp. Ricardo André
Naka

RESUMO

Este Trabalho de Conclusão de Curso apresenta o desenvolvimento do Core 4 ERP, uma aplicação web de gestão financeira pessoal concebida para apoiar o controle, a organização e a análise de informações relacionadas a contas, categorias, parceiros, contas correntes, cartões de crédito, assinaturas recorrentes, investimentos, notificações, conciliação bancária, relatórios financeiros e visualização consolidada por meio de *dashboard*. O sistema foi desenvolvido com *frontend* em *react*, *vite* e *tailwind css*, e *backend* em *java* utilizando *spring boot*, *spring security*, *spring data jpa*, *spring ai* e autenticação *jwt*, adotando o *postgresql* como banco de dados relacional. A proposta fundamenta-se na organização financeira do usuário por meio de uma arquitetura em camadas, baseada em *APIs rest*, isolamento de dados por usuário autenticado e persistência relacional com garantia de integridade referencial.

A metodologia aplicada contemplou as etapas de levantamento e análise de requisitos, modelagem conceitual e lógica de dados, definição da arquitetura da aplicação, implementação incremental dos módulos e validação funcional das principais rotas e telas do sistema. Foram adotadas boas práticas de engenharia de software, incluindo separação de responsabilidades, controle de acesso por perfis, padronização de camadas de serviço e repositório, auditoria automática de criação e modificação, *rate limiting* em rotas sensíveis e estratégias de segurança voltadas à proteção das informações financeiras dos usuários. A utilização do *spring security* associada à autenticação *jwt* contribuiu para o fortalecimento da segurança e da confiabilidade no acesso às funcionalidades disponibilizadas.

Como resultado, obteve-se uma solução funcional capaz de registrar, organizar, acompanhar e analisar informações financeiras pessoais de forma estruturada e centralizada. Entre os recursos implementados destacam-se autenticação e gerenciamento de usuários, controle de contas a pagar e a receber, baixa de contas com reflexo automático em saldo bancário, transferências entre contas correntes, gerenciamento de cartões de crédito com lançamentos, parcelamentos e fechamento de faturas, assinaturas recorrentes com geração automática de lançamentos, cadastro e acompanhamento de investimentos, conciliação bancária por importação de arquivos OFX, relatórios financeiros com DRE, fluxo de caixa, exportação em Excel e geração de PDF no frontend, além de assistente de IA para apoio a consultas e operações financeiras. Conclui-se que a aplicação desenvolvida contribui para o aprimoramento do controle orçamentário individual, promovendo maior autonomia, organização e suporte à tomada de decisão baseada em dados financeiros.

Palavras-chave: Gestão Financeira Pessoal. Engenharia de Software. *spring boot*. *postgresql*. *jwt*.

ABSTRACT

This Final Course Project presents the development of Core 4 ERP, a personal financial management web application designed to support the control, organization, and analysis of information related to accounts, categories, partners, bank accounts, credit cards, recurring subscriptions, investments, notifications, bank reconciliation, financial reports, and consolidated visualization through a dashboard. The system was developed with a frontend using React, Vite, and Tailwind CSS, and a backend in Java using Spring Boot, Spring Security, Spring Data JPA, Spring AI, and JWT authentication, adopting PostgreSQL as the relational database. The proposal is based on organizing users' financial information through a layered architecture, REST APIs, data isolation per authenticated user, and relational persistence with referential integrity.

The applied methodology included requirements elicitation and analysis, conceptual and logical data modeling, application architecture definition, incremental module implementation, and functional validation of the system's main routes and screens. Software engineering best practices were adopted, including separation of responsibilities, role-based access control, standardization of service and repository layers, automatic auditing of creation and modification data, rate limiting on sensitive routes, and security strategies aimed at protecting users' financial information. The use of Spring Security combined with JWT authentication contributed to strengthening security and reliability in accessing the available functionalities.

As a result, a functional solution capable of recording, organizing, monitoring, and analyzing personal financial information in a structured and centralized manner was obtained. The implemented features include user authentication and management, accounts payable and receivable control, settlement of accounts with automatic impact on bank balances, transfers between bank accounts, credit card management with transactions, installments, and invoice closing, recurring subscriptions with automatic transaction generation, investment registration and monitoring, bank reconciliation through OFX file import, financial reports with income statement, cash flow, Excel export, and PDF generation on the frontend, as well as an AI assistant to support financial queries and operations. It is concluded that the developed application contributes to improving individual budget control, promoting greater autonomy, organization, and support for data-driven financial decision-making.

Keywords: Personal Financial Management. Software Engineering. Spring Boot. PostgreSQL. JWT.

LISTA DE FIGURAS

Figura 1 - Taxa de erros 5xx por endpoint durante os testes com usuários.....	41
Figura 2 - Uso de memória heap da JVM durante a execução da aplicação.....	41
Figura 3 - Latência P95 por endpoint durante os testes com usuários.....	42
Figura 4 - Taxa de requisições por segundo durante os testes com usuários.....	42
Figura 5 - Tela de Login.....	53
Figura 6 - Dashboard / tela inicial.....	53
Figura 7 - Página de Parceiros.....	54
Figura 8 - Página de Categorias.....	55
Figura 9 - Página de Contas Correntes.....	55
Figura 10 - Página de Lançamentos.....	56
Figura 11 - Página de cartões.....	57
Figura 12 - Página de investimento.....	57
Figura 13 - Tela de Assinaturas.....	58
Figura 14 - Tela de Conciliação Bancária.....	59
Figura 15 - Tela de Configurações da conta.....	59
Figura 16 - Tela de Calendário financeiro.....	60
Figura 17 - Menu suspenso Lateral.....	61
Figura 18 - Diagrama de Arquitetura do Sistema.....	72
Figura 19 - Diagrama de Infraestrutura.....	72
Figura 20 - Diagrama de Classes (Entidades Java).....	72
Figura 21 - Diagrama de Sequência (Fluxo de Baixa/Conciliação de Conta).....	72
Figura 22 - Diagrama de Casos de Uso.....	72
Figura 23 - Modelo Entidade-Relacionamento (DER).....	72

LISTA DE TABELAS

Tabela 1 - Relação de participantes da pesquisa de satisfação.....	38
Tabela 2 - Resultado da avaliação NPS.....	34
Tabela 3 - Funcionalidades mais valorizadas pelos usuários.....	40
Tabela 4 - Resumo executivo todos os indicadores.....	43
Tabela 5 - Tipos declarados no backend Java.....	45
Tabela 6 - Distribuição por camada arquitetural MVC.....	45
Tabela 7 - <i>Endpoints</i> REST por método HTTP.....	46
Tabela 8 - Relação completa das 16 tabelas do banco de dados.....	46
Tabela 9 - Estimativa de complexidade ciclomática detalhada.....	47
Tabela 10 - Situação atual dos testes automatizados.....	48

LISTA DE GRÁFICOS

Gráfico 1 - Perfil dos participantes da pesquisa de satisfação.....	37
Gráfico 2 - Módulos utilizados com maior frequência.....	38
Gráfico 3 - Avaliação média dos aspectos do sistema.....	39

LISTA DE ABREVIATURAS E SIGLAS

- API** — APPLICATION PROGRAMMING INTERFACE (INTERFACE DE PROGRAMAÇÃO DE APLICAÇÕES)
- CNPJ** — CADASTRO NACIONAL DA PESSOA JURÍDICA
- CSV** — COMMA-SEPARATED VALUES (VALORES SEPARADOS POR VÍRGULAS)
- DER** — DIAGRAMA ENTIDADE-RELACIONAMENTO
- DRE** — DEMONSTRATIVO DE RESULTADOS DO EXERCÍCIO
- DTO** — DATA TRANSFER OBJECT (OBJETO DE TRANSFERÊNCIA DE DADOS)
- ERP** — ENTERPRISE RESOURCE PLANNING (PLANEJAMENTO DE RECURSOS EMPRESARIAIS)
- HTTP** — HYPERTEXT TRANSFER PROTOCOL (PROTOCOLO DE TRANSFERÊNCIA DE HIPERTEXTO)
- IA** — INTELIGÊNCIA ARTIFICIAL
- IP** — INTERNET PROTOCOL (PROTOCOLO DE INTERNET)
- JPA** — JAVA PERSISTENCE API
- JSON** — JAVASCRIPT OBJECT NOTATION (NOTAÇÃO DE OBJETOS JAVASCRIPT)
- JSONB** — JSON BINARY (JSON BINÁRIO)
- JWT** — JSON WEB TOKEN
- LGPD** — LEI GERAL DE PROTEÇÃO DE DADOS
- OFX** — OPEN FINANCIAL EXCHANGE (INTERCÂMBIO FINANCEIRO ABERTO)
- PDF** — PORTABLE DOCUMENT FORMAT (FORMATO DE DOCUMENTO PORTÁTIL)
- REST** — REPRESENTATIONAL STATE TRANSFER (TRANSFERÊNCIA DE ESTADO REPRESENTACIONAL)
- SPA** — SINGLE PAGE APPLICATION (APLICAÇÃO DE PÁGINA ÚNICA)
- SQL** — STRUCTURED QUERY LANGUAGE (LINGUAGEM DE CONSULTA ESTRUTURADA)
- UI** — USER INTERFACE (INTERFACE DO USUÁRIO)

SUMÁRIO

1. INTRODUÇÃO.....	10
1.1 Especificação de Requisitos.....	12
1.1.1 Requisitos Funcionais.....	13
1.1.2 Requisitos Não Funcionais.....	16
2. FUNDAMENTAÇÃO TEÓRICA.....	17
3. ARQUITETURA, EXPERIÊNCIA DO USUÁRIO E SEGURANÇA DO CORE 4 ERP.....	20
3.1 Padrão MVC e Arquitetura em Camadas.....	20
3.2 Experiência do Usuário e frontend.....	22
3.3 Segurança, Integridade e Observabilidade dos Dados.....	25
4. METODOLOGIA.....	27
4.1 Tecnologias Utilizadas.....	29
4.2 Etapas de Desenvolvimento.....	30
4.3 Validação e Testes.....	31
4.3.1 Validação e Monitoramento da Aplicação.....	33
4.3.2 Pesquisa de Satisfação com Usuários.....	34
4.3.3 Métricas de Engenharia de Software.....	43
5. DESCRIÇÃO DA ARQUITETURA DO SISTEMA E INTERFACE.....	49
5.1 Arquitetura do Sistema.....	49
5.2 Módulos Implementados.....	50
5.3 Diagramas Estruturais e Comportamentais.....	51
5.4 Interface do Sistema.....	52
6. MODELAGEM CONCEITUAL E FÍSICA.....	61
6.1 Principais Entidades.....	64
7. CONSIDERAÇÕES FINAIS.....	65
7.1 Trabalhos Futuros.....	67
REFERÊNCIAS BIBLIOGRÁFICAS.....	69
APÊNDICE A – DIAGRAMAS DO SISTEMA.....	72

1. INTRODUÇÃO

Além da relevância prática voltada à organização financeira pessoal, este trabalho apresenta contribuição científica para a área de Sistemas de Informação ao demonstrar a aplicação integrada de conceitos de Engenharia de Software, arquitetura de sistemas, segurança da informação, usabilidade e apoio à tomada de decisão em um sistema web de gestão financeira pessoal. A proposta contribui ao apresentar um modelo de solução que centraliza dados financeiros dispersos, organiza informações de receitas, despesas, contas, cartões, investimentos e compromissos futuros, e transforma esses registros em informações úteis por meio de *dashboards*, relatórios, conciliação bancária e recursos de inteligência artificial.

Do ponto de vista arquitetural, o sistema foi desenvolvido com base no padrão MVC, Model-View-Controller, que separa responsabilidades entre modelo, visão e controle, favorecendo a organização do código, a manutenção e a evolução da aplicação. Essa separação está alinhada aos princípios de qualidade de software discutidos por Sommerville (2011) e Pressman e Maxim (2016), segundo os quais a definição adequada da arquitetura contribui para reduzir a complexidade, melhorar a manutenibilidade e aumentar a confiabilidade do produto entregue. A adoção do MVC também se relaciona ao modelo de qualidade da ISO/IEC 25010, que considera características como manutenibilidade, segurança, usabilidade, confiabilidade e eficiência de desempenho como aspectos relevantes para avaliação da qualidade de sistemas de software.

No campo da segurança da informação, o trabalho contribui ao implementar autenticação e autorização por meio do Spring Security e de *tokens* JWT, reforçando o controle de acesso aos dados financeiros dos usuários. Essa decisão é relevante porque informações financeiras possuem caráter sensível e exigem mecanismos que restrinjam o acesso apenas ao usuário autenticado e autorizado. A documentação oficial do Spring Security destaca o suporte à proteção de endpoints por meio de *tokens bearer*, incluindo JWT, enquanto a OWASP aponta falhas de controle de acesso como uma das principais vulnerabilidades em aplicações web, com ampla ocorrência em sistemas avaliados.

A contribuição também se evidencia no aspecto de usabilidade, uma vez que o frontend foi projetado para facilitar a interação do usuário com informações financeiras complexas. A utilização de telas organizadas, dashboards, gráficos, relatórios e fluxos orientados reduz a necessidade de o usuário interpretar dados espalhados em planilhas, aplicativos bancários e anotações manuais. Esse ponto dialoga com Nielsen (1994), que defende princípios como visibilidade do estado do sistema, consistência, prevenção de

erros e reconhecimento em vez de memorização como fundamentos para interfaces mais compreensíveis e eficientes.

A relevância do tema é reforçada por estudos sobre educação financeira. Lusardi e Mitchell (2014) demonstram que o conhecimento financeiro influencia diretamente a capacidade de planejamento, poupança, endividamento e tomada de decisão dos indivíduos. No contexto brasileiro, a Pesquisa de Endividamento e Inadimplência do Consumidor, realizada pela CNC, indica que o nível de endividamento das famílias brasileiras permanece elevado, tendo alcançado 78,9% em dezembro de 2025, além de inadimplência de 29,4%. Esses dados reforçam a necessidade de soluções digitais que apoiem o controle financeiro e forneçam informações claras para decisões mais conscientes.

Os resultados obtidos na validação do Core 4 ERP reforçam sua contribuição aplicada. A pesquisa de satisfação contou com 34 respondentes, dos quais 67,6% utilizaram o sistema com foco em finanças pessoais. Os módulos Dashboard e Relatórios concentraram 55,9% das preferências de uso, indicando valorização das funcionalidades de visualização consolidada e análise financeira. Além disso, o sistema obteve NPS de 52,9, com 67,6% de usuários promotores, o que sugere aceitação positiva da solução. Na avaliação dos aspectos do sistema, interface e design alcançaram média 4,24, relatórios 4,21, conciliação OFX 3,94, desempenho 3,79 e experiência mobile 3,76, em escala de 1 a 5. As funcionalidades mais valorizadas foram dashboard completo, com 22 citações; design moderno, com 20; segurança dos dados, com 18; relatórios em Excel e PDF, com 14; conciliação bancária automática, com 14; chat de IA integrado, com 11; e calendário financeiro, também com 11 citações. Esses resultados indicam que os usuários perceberam valor justamente nos recursos associados à centralização, organização, análise e apoio à tomada de decisão financeira.

Dessa forma, a contribuição científica deste trabalho está em demonstrar como um sistema de informação web, estruturado por meio do padrão MVC, protegido por mecanismos de autenticação e autorização com Spring Security e JWT, e apoiado por uma interface voltada à usabilidade, pode ser utilizado como instrumento para reduzir a dispersão de informações financeiras, melhorar a organização dos registros pessoais e apoiar a tomada de decisão do usuário. Assim, o trabalho contribui para a área de Sistemas de Informação ao unir fundamentação teórica, implementação tecnológica e validação com usuários em torno de um problema real e socialmente relevante: a dificuldade de controle, análise e planejamento das finanças pessoais.

1.1 Especificação de Requisitos

A definição do escopo do Core 4 ERP parte da necessidade de transformar o problema identificado na introdução em uma solução computacional coerente com o contexto da gestão financeira pessoal. Para isso, a elicitação de requisitos assume papel fundamental, pois permite compreender as necessidades do usuário, delimitar as funcionalidades essenciais do sistema e estabelecer critérios técnicos que orientam a implementação, a validação e a manutenção da aplicação. Conforme Sommerville (2011), os requisitos representam a descrição dos serviços que o sistema deve oferecer e das restrições sob as quais deve operar, enquanto Pressman e Maxim (2016) destacam que a compreensão adequada do problema é uma etapa indispensável para que o software entregue corresponda às necessidades reais dos usuários. De forma complementar, Wiegers e Beatty (2013) afirmam que requisitos bem definidos reduzem ambiguidades, evitam retrabalho e servem como ponte entre as expectativas dos envolvidos e a solução a ser construída. Nesse sentido, a especificação dos requisitos deste trabalho foi orientada pela análise do problema de dispersão das informações financeiras pessoais, pela observação de funcionalidades presentes em sistemas ERP consolidados no mercado e pela adaptação desses princípios ao uso individual. Soluções como SAP ERP, que integra processos como finanças, vendas, compras, recursos humanos e cadeia de suprimentos em um sistema único (SAP, 2026), TOTVS Protheus, voltado à automação de rotinas administrativas e financeiras em diferentes áreas empresariais (TOTVS, 2026), Microsoft Dynamics 365 Finance, que enfatiza controle financeiro, automação, previsão de fluxo de caixa e apoio à decisão (MICROSOFT, 2026), e Oracle NetSuite, apresentado como uma plataforma de ERP em nuvem com relatórios financeiros, análise, CRM e comércio eletrônico integrados (Oracle Netsuite, 2026), demonstram a importância da centralização, da integração entre módulos e da confiabilidade dos dados em sistemas de gestão. Embora essas ferramentas sejam direcionadas principalmente ao ambiente organizacional, seus conceitos serviram como referência para compreender como processos financeiros podem ser estruturados, automatizados e analisados em uma base única de informações. Assim, o escopo do Core 4 ERP delimita-se ao desenvolvimento de uma aplicação web de gestão financeira pessoal capaz de integrar autenticação de usuários, categorias, parceiros, contas a pagar e a receber, contas correntes, transferências, cartões de crédito, faturas, assinaturas recorrentes, investimentos, notificações, calendário financeiro, conciliação bancária por arquivos OFX, relatórios financeiros, dashboard

consolidado, assistente de inteligência artificial, segurança, auditoria e monitoramento da aplicação. Dessa forma, os requisitos funcionais e não funcionais apresentados a seguir não constituem apenas uma lista técnica de recursos, mas representam a tradução do problema de pesquisa em funcionalidades, regras, restrições e atributos de qualidade necessários para que o sistema apoie a centralização das informações financeiras, reduza a fragmentação dos registros e ofereça ao usuário uma solução adequada para acompanhamento, análise e tomada de decisão.

1.1.1 Requisitos Funcionais

- RF001: O sistema deve permitir o cadastro, *login*, *logout* e consulta dos dados do usuário autenticado, garantindo que o acesso às funcionalidades internas ocorra somente após autenticação válida.
- RF002: O sistema deve permitir a atualização dos dados de perfil do usuário autenticado, mantendo as informações cadastrais vinculadas à sua conta.
- RF003: O sistema deve permitir o cadastro, listagem, edição, consulta e exclusão de categorias financeiras, possibilitando a organização das receitas, despesas e demais movimentações por tipo de classificação.
- RF004: O sistema deve permitir o cadastro, listagem, edição, consulta e exclusão de parceiros financeiros, como clientes, fornecedores, instituições ou demais entidades relacionadas às movimentações do usuário.
- RF005: O sistema deve permitir a consulta automática de dados empresariais por meio da BrasilAPI a partir do CNPJ informado, possibilitando o preenchimento automatizado de informações cadastrais e reduzindo inconsistências no cadastro de parceiros e empresas.
- RF006: O sistema deve permitir o cadastro, listagem, edição, parcelamento, consulta e exclusão de contas a pagar e a receber, associando cada registro financeiro ao usuário autenticado.
- RF007: O sistema deve permitir a baixa de contas financeiras, registrando data de pagamento, juros, multa, acréscimos, descontos e valor final, além de criar o respectivo registro de baixa, alterar o status da conta e ajustar o saldo da conta corrente vinculada.
- RF008: O sistema deve permitir o estorno de uma baixa financeira, removendo o registro de baixa, revertendo o saldo da conta corrente associada e retornando a conta ao status pendente.

- RF009: O sistema deve permitir o cadastro, listagem, edição, consulta e exclusão de contas correntes, mantendo os saldos atualizados conforme as movimentações realizadas no sistema.
- RF010: O sistema deve permitir transferências entre contas correntes do mesmo usuário, validando conta de origem e destino, impedindo transferências entre contas iguais e atualizando os saldos envolvidos na operação.
- RF011: O sistema deve permitir o cadastro, listagem, edição, consulta e exclusão de cartões de crédito associados ao usuário autenticado.
- RF012: O sistema deve permitir o cadastro, listagem, edição e exclusão de lançamentos de cartão de crédito, incluindo lançamentos à vista e parcelados.
- RF013: O sistema deve permitir que lançamentos parcelados de cartão de crédito dividam o valor total entre as parcelas ou repitam o valor informado em cada parcela, conforme a configuração definida na operação.
- RF014: O sistema deve permitir o fechamento de faturas de cartão de crédito, somando os lançamentos do período, gerando uma conta a pagar correspondente e marcando a fatura como fechada.
- RF015: O sistema deve permitir o cadastro, listagem, edição, consulta e exclusão de contas de investimento.
- RF016: O sistema deve permitir o cadastro, listagem, edição e exclusão de tipos personalizados de investimento definidos pelo usuário.
- RF017: O sistema deve permitir o registro e a consulta de transações de investimento, associando cada movimentação à respectiva conta de investimento e, quando aplicável, à conta corrente de origem.
- RF018: O sistema deve exibir um dashboard financeiro consolidado com saldos, contas, cartões, investimentos, indicadores e informações resumidas para apoiar a análise da situação financeira do usuário.
- RF019: O sistema deve permitir a consulta, marcação como lida e sincronização de notificações financeiras do usuário autenticado.
- RF020: O sistema deve sincronizar automaticamente notificações de vencimento, verificando contas pendentes com data anterior ao dia corrente, atualizando seu status para “ATRASADO” e criando notificações apenas quando ainda inexistentes, garantindo idempotência na geração dos registros.

- RF021: O sistema deve gerar notificações automáticas para cartões de crédito cuja data de fechamento coincida com o dia atual, alertando o usuário sobre o fechamento da fatura.
- RF022: O sistema deve permitir o cadastro, listagem, edição, consulta e exclusão de assinaturas recorrentes.
- RF023: O sistema deve gerar automaticamente lançamentos de cartão relacionados a assinaturas recorrentes ativas por meio de processo agendado diariamente, eliminando a necessidade de criação manual desses lançamentos pelo usuário.
- RF024: O sistema deve permitir a importação de arquivos OFX para conciliação bancária, processando as transações do extrato e sugerindo vínculos com contas financeiras existentes.
- RF025: O sistema deve permitir, durante a conciliação bancária, vincular transações importadas a contas existentes, criar novas contas a partir dos itens conciliados, realizar baixas, ignorar transações, desfazer ações e finalizar a conciliação.
- RF026: O sistema deve permitir a geração de relatórios financeiros, incluindo fluxo de caixa, DRE, extrato, contas abertas, investimentos, cartões, posição financeira e assinaturas.
- RF027: O sistema deve disponibilizar relatórios financeiros em formato json para visualização na aplicação, em planilhas Excel geradas no backend por meio do Apache POI e em PDF gerado no frontend por meio do jsPDF.
- RF028: O sistema deve disponibilizar um assistente de inteligência artificial para consultas financeiras, apoio a lançamentos e geração de relatórios, integrado aos dados financeiros do usuário.
- RF029: O sistema deve permitir recuperação e redefinição de senha por meio de *token* temporário persistido no usuário, envio de link por e-mail e validação do token antes da alteração da senha.
- RF030: O sistema deve controlar tentativas malsucedidas de login por usuário, registrando a quantidade de falhas consecutivas e bloqueando temporariamente a conta quando o limite definido pela aplicação for atingido.
- RF031: O sistema deve exibir um calendário financeiro mensal com contas a pagar, contas a receber e assinaturas organizadas por data de vencimento, permitindo a realização de baixas diretamente pelo calendário.

1.1.2 Requisitos Não Funcionais

- RNF001: O sistema deve utilizar autenticação baseada em JWT, com suporte a armazenamento do token em *cookie* HttpOnly e envio por bearer Token, garantindo maior segurança no processo de autenticação e reduzindo riscos de exposição indevida das credenciais em rotas protegidas.
- RNF002: O backend deve ser implementado em Java com Spring Boot, Spring Security e Spring Data JPA, favorecendo a organização da aplicação em camadas, a padronização das regras de segurança, a exposição de APIs REST e a persistência estruturada das entidades do domínio financeiro.
- RNF003: O banco de dados deve utilizar PostgreSQL como sistema gerenciador relacional, assegurando integridade referencial entre usuários, categorias, parceiros, contas, contas correntes, cartões, faturas, assinaturas, investimentos, conciliações e demais entidades financeiras relacionadas.
- RNF004: As operações envolvendo dados financeiros devem ser isoladas por usuário autenticado, utilizando filtros por *usuarioid* nas consultas e operações de persistência, a fim de impedir acesso indevido a registros pertencentes a outros usuários.
- RNF005: A interface do sistema deve ser responsiva, protegida por rotas autenticadas e construída com componentes reutilizáveis, garantindo melhor experiência de uso em diferentes tamanhos de tela e mantendo consistência visual e funcional entre os módulos.
- RNF006: O sistema deve expor documentação interativa da API por meio de Swagger/OpenAPI, permitindo consulta, entendimento e teste dos endpoints disponíveis durante o desenvolvimento, validação e manutenção da aplicação.
- RNF007: As entidades de domínio devem registrar metadados básicos de auditoria, como data e usuário de criação e última modificação, por meio de JPA Auditing, possibilitando maior rastreabilidade das operações realizadas no sistema.
- RNF008: O sistema deve aplicar rate limiting por IP em rotas sensíveis, como login, registro, chat e *upload* OFX, utilizando Bucket4j para mitigar tentativas de força bruta, abuso de API e uso excessivo de recursos da aplicação.
- RNF009: O sistema deve bloquear temporariamente contas de usuário após múltiplas tentativas de login malsucedidas, registrando a quantidade de falhas

consecutivas no campo *login_attempts* e o período de bloqueio no campo *locked_until*, complementando a proteção por IP com uma regra de segurança aplicada diretamente à conta do usuário.

- RNF010: O sistema deve expor métricas de saúde e desempenho por meio do Spring Boot Actuator e Prometheus, oferecendo suporte à observabilidade da aplicação em ambiente de produção e permitindo acompanhar disponibilidade, comportamento das requisições e indicadores operacionais.
- RNF011: O sistema deve proteger rotas privadas por meio de autenticação stateless, liberando acesso público apenas a endpoints necessários, como login, registro, recuperação de senha, documentação Swagger e *health check*.
- RNF012: O sistema deve manter registros de auditoria e monitoramento sem comprometer o desempenho das operações principais, utilizando mecanismos de observabilidade e persistência de *logs* compatíveis com o funcionamento da aplicação.

2. FUNDAMENTAÇÃO TEÓRICA

A gestão financeira pessoal envolve um conjunto de conhecimentos, comportamentos, decisões e ferramentas que permitem ao indivíduo organizar sua vida econômica, acompanhar receitas e despesas, planejar compromissos futuros, utilizar crédito de forma consciente e tomar decisões com base em informações confiáveis. Nesse contexto, a educação financeira, a alfabetização financeira, o comportamento financeiro e o planejamento financeiro constituem conceitos relacionados, mas não equivalentes. A educação financeira pode ser compreendida como o processo pelo qual indivíduos desenvolvem compreensão sobre produtos, conceitos e riscos financeiros, adquirindo habilidades e confiança para realizar escolhas informadas e adotar ações capazes de melhorar seu bem-estar financeiro (OECD, 2005). Já a alfabetização financeira refere-se à combinação de conhecimento, consciência, habilidades, atitudes e comportamentos necessários para tomar decisões financeiras adequadas e alcançar bem-estar financeiro individual (OECD, 2023). Assim, enquanto a educação financeira está associada ao processo de formação e aprendizagem, a alfabetização financeira representa a capacidade prática de aplicar esse conhecimento em situações reais.

Essa distinção é importante porque conhecer conceitos financeiros não garante, isoladamente, uma gestão financeira eficiente. O comportamento financeiro corresponde à forma como o indivíduo utiliza, na prática, seus recursos financeiros, incluindo hábitos

de consumo, organização de pagamentos, uso do crédito, poupança, investimento e acompanhamento de dívidas. A OECD/INFE (2023) destaca que a mensuração da alfabetização financeira deve considerar não apenas conhecimento financeiro, mas também comportamento e atitudes financeiras, pois a tomada de decisão depende tanto da compreensão dos conceitos quanto da forma como o indivíduo age diante de oportunidades, riscos e limitações econômicas. Dessa forma, uma pessoa pode compreender noções como juros, inflação e diversificação, mas ainda assim apresentar dificuldades caso não acompanhe seus gastos, não registre compromissos futuros ou não possua instrumentos adequados para visualizar sua situação financeira.

Lusardi e Mitchell (2014) reforçam essa relação ao demonstrar que a alfabetização financeira influencia diretamente decisões associadas ao planejamento, à poupança, ao endividamento, à aposentadoria e à acumulação de riqueza. Para as autoras, indivíduos com maior domínio de conhecimentos financeiros tendem a tomar decisões mais consistentes ao longo do tempo, enquanto baixos níveis de alfabetização financeira estão associados a maior vulnerabilidade diante de produtos financeiros, crédito de alto custo e ausência de planejamento de longo prazo. Esse cenário também aparece em levantamentos internacionais. A S&P Global Financial Literacy Survey, realizada em parceria com Gallup, World Bank Development Research Group e Global Financial Literacy Excellence Center, indicou que apenas 35% dos adultos brasileiros eram considerados financeiramente alfabetizados em 2014, percentual que evidencia limitações relevantes na compreensão de conceitos financeiros básicos no país (GFLEC, 2014).

No contexto brasileiro, a relevância do tema também se relaciona ao endividamento e à dificuldade de organização financeira. O Banco Central do Brasil (2020), ao tratar do endividamento de risco, considera como situação crítica aquela em que o volume de dívida supera a capacidade de pagamento do indivíduo, prejudicando o gerenciamento dos recursos financeiros e a qualidade de vida. Segundo o estudo, em dezembro de 2019 havia 4,6 milhões de tomadores em situação de endividamento de risco, correspondendo a 5,4% da população com crédito ativo no Sistema Financeiro Nacional analisada pelo relatório. Esses dados demonstram que o problema financeiro não está apenas na existência de dívidas, mas na ausência de controle, previsibilidade e capacidade de acompanhar compromissos, saldos, vencimentos e impactos futuros das decisões tomadas no presente.

O planejamento financeiro pessoal, nesse sentido, atua como uma prática complementar à educação e à alfabetização financeira. Planejar financeiramente significa organizar informações sobre receitas, despesas, dívidas, investimentos, patrimônio, compromissos recorrentes e objetivos futuros, permitindo que o indivíduo compreenda sua situação econômica e tome decisões mais adequadas. Marion (2018) destaca que a contabilidade permite registrar, organizar e interpretar informações econômicas, servindo como apoio à análise e à tomada de decisão. Embora a contabilidade seja tradicionalmente aplicada ao contexto empresarial, seus conceitos básicos, como receitas, despesas, patrimônio, resultado, fluxo de caixa e posição financeira, também são relevantes para a vida pessoal, pois ajudam o usuário a interpretar sua realidade financeira de forma estruturada. No Core 4 ERP, esses conceitos aparecem na organização de contas a pagar e a receber, contas correntes, cartões de crédito, investimentos, fluxo de caixa, DRE, posição financeira, relatórios e dashboard consolidado.

A utilização de sistemas de informação amplia essa capacidade de organização ao transformar registros dispersos em informações acessíveis, integradas e analisáveis. O'Brien e Marakas (2013) destacam que sistemas de informação possibilitam coletar, processar, armazenar e distribuir dados para apoiar o controle e a tomada de decisão. Laudon e Laudon (2020) também ressaltam que sistemas integrados contribuem para melhorar o monitoramento das informações e reduzir a fragmentação dos processos. Aplicado à gestão financeira pessoal, esse princípio permite compreender que o problema não está apenas na falta de dados, mas na dificuldade de reunir e interpretar informações que normalmente estão espalhadas em aplicativos bancários, faturas, planilhas, anotações manuais, extratos, comprovantes e ferramentas desconectadas.

Nesse ponto, os sistemas ERP oferecem uma base conceitual relevante para o desenvolvimento de soluções integradas. Os sistemas Enterprise Resource Planning caracterizam-se pela integração de informações provenientes de diferentes áreas em uma base única, favorecendo padronização, rastreabilidade, consistência dos dados e apoio à gestão. Bastos (2024) destaca que os ERPs contribuem para centralizar e confiabilizar informações organizacionais, reduzindo redundâncias e melhorando o acompanhamento dos processos. Soluções consolidadas no mercado, como SAP ERP, TOTVS Protheus, Microsoft Dynamics 365 Finance e Oracle NetSuite, exemplificam essa lógica ao integrar módulos financeiros, relatórios, processos administrativos, automação, análise de desempenho e suporte à tomada de decisão em ambientes empresariais (SAP, 2026; TOTVS, 2026; MICROSOFT, 2026; ORACLE NETSUITE, 2026). Embora essas

plataformas sejam voltadas principalmente a organizações, seus princípios de integração, controle, automação e confiabilidade podem ser adaptados ao contexto da gestão financeira pessoal.

A aplicação de conceitos de ERP ao contexto pessoal justifica-se porque o indivíduo também lida com processos financeiros diversos, ainda que em menor escala. Contas a pagar e a receber, cartões de crédito, contas correntes, assinaturas recorrentes, investimentos, parceiros financeiros, vencimentos, transferências, conciliações bancárias e relatórios representam informações que precisam ser registradas, relacionadas e analisadas de forma integrada. Quando esses dados permanecem separados, o usuário tende a depender de esforço manual para compreender sua situação financeira, o que aumenta a possibilidade de esquecimentos, inconsistências, atrasos e decisões baseadas em informações incompletas. Dessa forma, a lógica ERP, ao ser adaptada para o uso individual, permite estruturar a gestão financeira pessoal como um conjunto de módulos interligados em uma mesma base de dados.

No Core 4 ERP, essa fundamentação teórica sustenta a proposta de uma aplicação web voltada à centralização, organização e análise das informações financeiras do usuário. O sistema incorpora conceitos de educação financeira e planejamento ao oferecer recursos que favorecem a visualização de saldos, vencimentos, despesas, receitas, investimentos e compromissos futuros. Também aplica princípios de sistemas ERP ao integrar módulos de contas, cartões, categorias, parceiros, contas correntes, assinaturas, investimentos, conciliação bancária, relatórios, dashboard, calendário financeiro, notificações e assistente de inteligência artificial. Assim, o sistema não se limita ao registro operacional de dados, mas busca oferecer uma estrutura de apoio à tomada de decisão, permitindo que o usuário acompanhe sua situação financeira de forma mais clara, integrada e consistente.

3. ARQUITETURA, EXPERIÊNCIA DO USUÁRIO E SEGURANÇA DO CORE 4 ERP

3.1 Padrão MVC e Arquitetura em Camadas

A construção do CORE 4 ERP fundamenta-se em princípios de Engenharia de Software voltados à organização, manutenibilidade, segurança e evolução incremental do sistema. Segundo Pressman e Maxim (2016), a Engenharia de Software fornece métodos e práticas capazes de reduzir a complexidade do desenvolvimento e aumentar a qualidade

do produto entregue. De forma complementar, Sommerville (2011) destaca que decisões arquiteturais influenciam diretamente atributos como confiabilidade, manutenção, desempenho, evolução e organização estrutural. No contexto deste trabalho, esses princípios foram aplicados por meio de uma arquitetura em camadas, estruturada para separar responsabilidades entre interface, comunicação com a API, regras de negócio, persistência e banco de dados.

A arquitetura adotada aproxima-se do padrão Model-View-Controller, MVC, adaptado ao contexto de uma aplicação web moderna com frontend desacoplado e backend orientado a APIs REST. No modelo clássico, o padrão MVC separa a representação dos dados, a interface de apresentação e o controle das interações do usuário. No Core 4 ERP, essa separação ocorre de forma distribuída: a camada de visualização é representada pelo frontend em React; a camada de controle é composta pelos *controllers* do backend em Spring Boot, responsáveis por receber requisições HTTP, validar entradas e direcionar chamadas aos serviços; e a camada de modelo envolve entidades JPA, DTOs, *repositories* e estruturas persistidas no PostgreSQL. Essa organização reduz o acoplamento entre as partes do sistema e permite que cada camada evolua com responsabilidades mais claras.

A documentação oficial do Spring Web MVC destaca que o *framework* é organizado em torno do DispatcherServlet, responsável por encaminhar requisições aos manipuladores adequados, o que reforça a lógica de controle centralizado e separação de responsabilidades no backend (SPRING, 2026). No Core 4 ERP, essa estrutura aparece nos módulos organizados por domínio, como *auth*, *conta*, *contaCorrente*, *cartaoCredito*, *investimento*, *assinatura*, *conciliacao*, *relatorio*, *dashboard*, *notificacao* e *chat*. Em cada módulo, os controllers expõem endpoints REST, os *services* concentram regras de negócio e os *repositories* realizam a comunicação com o banco de dados por meio do Spring Data JPA. Essa divisão torna o sistema mais compreensível, facilita testes e reduz o risco de que alterações em uma funcionalidade afetem indevidamente outras partes da aplicação.

A separação em camadas também contribui para a rastreabilidade das regras de negócio. Operações como baixa de contas, estorno, fechamento de fatura, transferência entre contas correntes, registro de investimentos, geração de notificações, conciliação bancária e emissão de relatórios não ficam dispersas diretamente na interface ou no acesso ao banco. Essas regras são concentradas na camada de serviço, permitindo que o frontend apenas acione operações e apresente resultados ao usuário. Essa decisão é importante

porque sistemas financeiros dependem de consistência operacional: uma baixa de conta, por exemplo, não representa apenas uma alteração visual, mas uma operação que cria registro de pagamento, altera status da conta e atualiza saldo de conta corrente.

O Diagrama de Arquitetura do Sistema, apresentado no Apêndice A, Figura 18, sintetiza essa organização ao demonstrar a separação entre frontend, backend, banco de dados e serviços auxiliares. Esse diagrama evidencia que o Core 4 ERP não foi estruturado como um conjunto isolado de telas, mas como uma aplicação distribuída em camadas, na qual cada componente possui função definida dentro do fluxo geral do sistema. Já o Diagrama de Infraestrutura, apresentado no Apêndice A, Figura 19, complementa essa visão ao representar os elementos envolvidos na execução da aplicação, incluindo ambiente de frontend, API, banco de dados, observabilidade e comunicação entre serviços. Esses diagramas são relevantes porque documentam a arquitetura sob duas perspectivas: a lógica interna da aplicação e a disposição dos componentes em ambiente de execução.

A modelagem estrutural também está documentada no Diagrama de Classes, apresentado no Apêndice A, Figura 20. Esse diagrama evidencia as principais entidades Java do sistema e suas relações, demonstrando como conceitos do domínio financeiro foram transformados em estruturas computacionais. Entidades como usuário, conta, conta baixada, conta corrente, cartão de crédito, fatura, lançamento, investimento, assinatura, conciliação e notificação representam elementos centrais do problema de gestão financeira pessoal. A existência dessas entidades confirma a aplicação do princípio de modelagem orientada ao domínio, no qual os objetos do sistema refletem os elementos reais que precisam ser registrados, manipulados e analisados.

Além da estrutura estática, o comportamento das operações críticas é representado pelo Diagrama de Sequência do fluxo de baixa e conciliação de conta, apresentado no Apêndice A, Figura 21. Esse tipo de diagrama é importante porque demonstra a ordem das interações entre usuário, interface, controller, *service*, *repository* e banco de dados durante uma operação financeira. No caso do Core 4 ERP, esse fluxo evidencia que ações como baixar uma conta ou conciliar uma movimentação bancária envolvem validações, persistência, alteração de status e atualização de saldos. Assim, os diagramas não atuam apenas como ilustrações, mas como instrumentos de documentação técnica que reforçam a coerência entre requisitos, arquitetura e implementação.

3.2 Experiência do Usuário e FRONTEND

A experiência do usuário constitui um aspecto essencial em sistemas de gestão financeira pessoal, pois a utilidade da aplicação depende não apenas da existência das funcionalidades, mas da capacidade de o usuário compreender, registrar, consultar e interpretar informações financeiras de forma clara. A norma ISO 9241-11 relaciona usabilidade à possibilidade de usuários específicos alcançarem objetivos com efetividade, eficiência e satisfação em determinado contexto de uso (ISO, 2018). No caso do Core 4 ERP, esse contexto envolve tarefas recorrentes, como registrar contas, acompanhar vencimentos, consultar saldos, verificar faturas, analisar relatórios, acompanhar investimentos e interpretar indicadores financeiros.

Nielsen (1994) destaca que interfaces devem considerar princípios como visibilidade do estado do sistema, consistência, prevenção de erros, reconhecimento em vez de memorização e *feedback* adequado. Esses princípios são especialmente relevantes em uma aplicação financeira, pois erros de interpretação podem gerar consequências práticas, como atrasos em pagamentos, saldos incorretos, registros duplicados ou decisões baseadas em informações incompletas. Por esse motivo, o frontend do Core 4 ERP foi desenvolvido para organizar os módulos de forma visualmente consistente, oferecendo navegação lateral, componentes reutilizáveis, tabelas, filtros, indicadores, gráficos, cards informativos, modais de confirmação, mensagens de erro e alertas de operação.

A escolha do React como tecnologia de frontend está alinhada à necessidade de construir uma interface modular e reutilizável. A documentação oficial do React enfatiza a composição por componentes e o gerenciamento de estado como fundamentos para criação de interfaces interativas (REACT, 2026). No Core 4 ERP, essa abordagem aparece na divisão entre páginas, componentes de *layout*, componentes de interface, *hooks*, contexto de autenticação e biblioteca de comunicação com a API. Telas como Dashboard, Contas Financeiras, Cartões, Investimentos, Conciliação, Relatórios, Calendário, Notificações e Configurações compartilham padrões visuais e funcionais, reduzindo a curva de aprendizado do usuário.

O uso de componentes reutilizáveis contribui para a consistência da experiência. Elementos como cabeçalhos de página, campos de formulário, tabelas de dados, botões de ação, modais de confirmação, estados vazios, notificações visuais e componentes de relatório permitem que diferentes módulos mantenham comportamento semelhante. Essa uniformidade favorece o princípio de reconhecimento em vez de memorização, pois o usuário passa a compreender a lógica da aplicação e a aplicar esse aprendizado em novas telas. Em vez de aprender um fluxo completamente diferente a cada módulo, ele encontra

padrões recorrentes de cadastro, listagem, edição, exclusão, filtro, confirmação e visualização.

O Tailwind CSS foi utilizado para estruturar a camada visual da aplicação com classes utilitárias, favorecendo padronização, responsividade e velocidade de desenvolvimento. Em conjunto com componentes próprios e ícones Lucide, a interface busca comunicar ações e estados de forma mais objetiva. Em sistemas financeiros, ícones, cores, agrupamentos e hierarquia visual têm papel importante porque ajudam o usuário a diferenciar ações de consulta, edição, exclusão, baixa, estorno, exportação, conciliação e alerta. Essa organização visual reduz a carga cognitiva e facilita o acompanhamento de informações que, quando apresentadas de forma dispersa, poderiam se tornar difíceis de interpretar.

O Dashboard representa um dos principais pontos de aplicação dos princípios de usabilidade no sistema. Nele, informações de contas, cartões, investimentos, saldos, despesas, receitas e indicadores são consolidadas para oferecer uma visão geral da situação financeira do usuário. Essa tela materializa a proposta do sistema de transformar dados financeiros em informações úteis para decisão. Em vez de exigir que o usuário acesse cada módulo separadamente para compreender sua posição financeira, o dashboard apresenta uma síntese visual do estado atual, favorecendo análise rápida e acompanhamento recorrente.

A conciliação bancária também evidencia a importância da experiência do usuário. A importação de arquivos OFX poderia ser uma operação tecnicamente complexa, pois envolve leitura de extrato, identificação de transações, comparação com contas existentes, sugestão de vínculos, criação de novos registros, baixa de contas e finalização da conciliação. No Core 4 ERP, esse processo é apresentado por meio de interface orientada, permitindo que o usuário visualize os itens importados e escolha ações como vincular, criar conta, ignorar, desfazer ou finalizar. Essa abordagem reduz a complexidade percebida e aproxima o usuário de um fluxo guiado, coerente com o princípio de prevenção de erros.

O Diagrama de Casos de Uso, apresentado no Apêndice A, Figura 22, conecta-se diretamente à experiência do usuário, pois descreve as principais interações entre o ator e o sistema. Esse diagrama demonstra que a interface foi pensada a partir das ações esperadas do usuário, como autenticar-se, gerenciar contas, consultar dashboard, realizar conciliações, gerar relatórios, acompanhar notificações, administrar cartões, registrar investimentos e utilizar o assistente de IA. Assim, o frontend não é apenas uma camada

visual sobre o backend, mas a materialização dos casos de uso levantados durante a especificação do sistema.

3.3 Segurança, Integridade e Observabilidade dos Dados

A segurança da informação é um requisito central em sistemas financeiros, pois a aplicação manipula dados sensíveis relacionados a receitas, despesas, contas bancárias, cartões, investimentos, parceiros, faturas e histórico de movimentações. Stallings (2015) destaca que autenticação, autorização, controle de acesso, criptografia e proteção de credenciais são mecanismos fundamentais para reduzir riscos de acesso indevido e comprometimento de dados. No Core 4 ERP, esses princípios são aplicados por meio de autenticação baseada em JWT, armazenamento de senhas por *hash*, filtros de segurança, controle de rotas protegidas, isolamento lógico por usuário autenticado, rate limiting e bloqueio temporário de contas após múltiplas tentativas de login malsucedidas.

O uso de JWT está alinhado ao padrão definido pela RFC 7519, que descreve o JSON Web Token como um formato compacto para representar declarações entre partes de forma segura, podendo ser assinado e verificado (JONES; BRADLEY; SAKIMURA, 2015). No sistema, o JWT é utilizado para identificar o usuário autenticado e proteger as rotas privadas. A adoção de autenticação *stateless* permite que cada requisição carregue as informações necessárias para validação do acesso, sem depender de sessão mantida no servidor. Essa característica é adequada para APIs REST e contribui para a escalabilidade e separação entre frontend e backend.

O controle de acesso é reforçado pelo isolamento dos registros financeiros por *usuarioid*. Essa decisão é essencial porque, em um sistema multiusuário, autenticar o usuário não basta; também é necessário garantir que cada consulta, alteração ou exclusão seja limitada aos dados pertencentes ao usuário autenticado. A OWASP classifica falhas de controle de acesso como uma das categorias mais críticas em aplicações web, destacando que esse tipo de vulnerabilidade pode permitir divulgação, modificação ou exclusão indevida de dados (OWASP, 2021). No Core 4 ERP, a filtragem por usuário nos repositories e services reduz o risco de exposição cruzada de dados entre contas diferentes.

Além da autenticação e autorização, o sistema implementa mecanismos de proteção contra abuso de API. Rotas sensíveis, como login, registro, chat e upload OFX, utilizam rate limiting por IP com Bucket4j, biblioteca baseada no algoritmo *token bucket*. Essa proteção limita a quantidade de requisições em determinado intervalo de tempo,

reduzindo riscos associados a força bruta, automação excessiva e consumo indevido de recursos. O bloqueio temporário de conta após múltiplas falhas de login complementa essa proteção, pois atua diretamente sobre a conta do usuário por meio dos campos *login_attempts* e *locked_until*, impedindo novas autenticações até o término do período de bloqueio.

A proteção dos dados também se relaciona à Lei Geral de Proteção de Dados Pessoais, Lei nº 13.709/2018, que estabelece princípios para o tratamento de dados pessoais no Brasil. Embora o Core 4 ERP seja um projeto acadêmico, sua natureza financeira exige atenção a princípios como segurança, prevenção, necessidade e responsabilização. O isolamento lógico dos registros, o controle de acesso, a autenticação segura e a redução de exposição indevida de informações contribuem para uma abordagem mais adequada ao tratamento de dados pessoais e financeiros.

A integridade dos dados é assegurada pela combinação entre PostgreSQL, JPA, transações e modelagem relacional. A documentação do PostgreSQL destaca que chaves primárias, chaves estrangeiras, restrições de unicidade e demais constraints permitem controlar a validade dos dados armazenados e manter a integridade referencial entre tabelas (POSTGRESQL, 2026). No Core 4 ERP, essa estrutura é fundamental para relacionar usuários, contas, contas baixadas, contas correntes, cartões, faturas, lançamentos, investimentos, assinaturas, conciliações e notificações. O Modelo Entidade-Relacionamento, apresentado no Apêndice A, Figura 23, documenta essa estrutura e demonstra como as entidades financeiras se relacionam no banco de dados.

A consistência transacional é especialmente importante nas operações financeiras. Baixar uma conta, estornar um pagamento, transferir valores entre contas correntes, fechar uma fatura ou finalizar uma conciliação bancária são operações que não podem ser executadas parcialmente. Caso uma etapa seja concluída e outra falhe, o sistema pode gerar saldo incorreto, fatura inconsistente ou registro financeiro incompleto. Por isso, a utilização de serviços transacionais no backend, em conjunto com a persistência relacional, contribui para que essas operações preservem a coerência dos dados.

Outro ponto relevante é a auditoria. O Spring Data JPA oferece suporte a metadados de auditoria, como usuário e data de criação ou modificação de registros (SPRING DATA JPA, 2026). No Core 4 ERP, entidades de domínio herdam campos de auditoria, o que permite registrar informações básicas sobre o ciclo de vida dos dados. Essa prática favorece rastreabilidade, manutenção e análise posterior de alterações, especialmente em um sistema que lida com registros financeiros e operações recorrentes.

Por fim, a observabilidade complementa a segurança e a confiabilidade da aplicação ao permitir acompanhamento de saúde, desempenho e comportamento operacional. O Spring Boot Actuator disponibiliza endpoints de monitoramento, incluindo métricas e integração com Prometheus (SPRING BOOT, 2026). No projeto, essa estrutura aparece na exposição de métricas, logs e recursos de monitoramento, permitindo avaliar disponibilidade, desempenho de requisições e possíveis falhas em ambiente de execução. Dessa forma, a segurança do Core 4 ERP não se limita ao login, mas envolve um conjunto de práticas que inclui autenticação, autorização, isolamento de dados, integridade referencial, auditoria, prevenção de abuso e monitoramento.

4. METODOLOGIA

A presente pesquisa classifica-se como aplicada, uma vez que tem como finalidade o desenvolvimento de uma solução computacional voltada à resolução de um problema prático: a dificuldade de centralização, organização e análise de informações financeiras pessoais. Segundo Gil (2022), a pesquisa aplicada busca gerar conhecimentos direcionados à solução de problemas concretos, aproximando a investigação acadêmica de uma necessidade real observada em determinado contexto. Nesse sentido, este trabalho não se limita à discussão teórica sobre gestão financeira pessoal, sistemas ERP ou Engenharia de Software, mas utiliza esses fundamentos como base para projetar, implementar e validar o Core 4 ERP, uma aplicação web destinada ao gerenciamento financeiro pessoal.

Quanto à abordagem, o trabalho possui caráter qualitativo e exploratório, pois envolve a compreensão do domínio de gestão financeira pessoal, a identificação de necessidades dos usuários, a análise de funcionalidades presentes em sistemas de gestão, a definição de requisitos, a modelagem do sistema e a avaliação prática da solução desenvolvida. A pesquisa exploratória justifica-se porque o projeto busca compreender como princípios tradicionalmente associados a sistemas ERP empresariais podem ser adaptados ao contexto individual, considerando módulos como contas a pagar e receber, cartões de crédito, contas correntes, investimentos, assinaturas recorrentes, conciliação bancária, relatórios, calendário financeiro, notificações e assistente de inteligência artificial. Dessa forma, a investigação parte de um problema amplo, relacionado à fragmentação das informações financeiras pessoais, e o transforma progressivamente em requisitos, modelos, funcionalidades e critérios de validação.

Como procedimento técnico, adotou-se o desenvolvimento iterativo e incremental. Esse modelo foi escolhido porque o Core 4 ERP é composto por diversos módulos interdependentes, cuja implementação exigiu evolução progressiva, validação contínua e refinamento das regras de negócio ao longo do desenvolvimento. Conforme Sommerville (2011), abordagens incrementais permitem que partes do sistema sejam entregues e avaliadas gradualmente, reduzindo riscos e possibilitando ajustes conforme novos requisitos ou necessidades são identificados. Pressman e Maxim (2016) também destacam que modelos iterativos favorecem o tratamento da complexidade, pois permitem construir o software em ciclos sucessivos, nos quais análise, projeto, implementação e testes podem ser retomados à medida que o produto evolui.

A adoção desse modelo mostrou-se adequada ao escopo do Core 4 ERP porque os módulos financeiros possuem dependências funcionais entre si. A implementação de contas a pagar e a receber, por exemplo, exigiu posteriormente regras de baixa, estorno e atualização de saldo em conta corrente. Da mesma forma, o módulo de cartões de crédito foi evoluído para contemplar lançamentos, parcelamentos, faturas e fechamento de fatura com geração automática de conta a pagar. O módulo de conciliação bancária, por sua vez, dependeu da existência prévia de contas financeiras e contas correntes para permitir importação de arquivos OFX, sugestão de vínculos, criação de contas, baixa de registros e finalização da conciliação. Assim, a construção incremental permitiu que cada funcionalidade fosse desenvolvida, testada e ajustada antes de ser integrada a fluxos mais complexos.

Além disso, o desenvolvimento iterativo favoreceu a evolução da arquitetura do sistema. Inicialmente, foram definidos os módulos centrais de autenticação, usuários, categorias, parceiros e contas financeiras. Em seguida, foram incorporadas funcionalidades complementares, como contas correntes, cartões de crédito, investimentos, assinaturas recorrentes, notificações, relatórios, dashboard, calendário financeiro e assistente de IA. Essa progressão permitiu que a aplicação mantivesse coerência estrutural, pois cada novo módulo pôde reaproveitar padrões já estabelecidos, como controllers rest, services para regras de negócio, repositories JPA, DTOs de entrada e saída, validações, tratamento centralizado de exceções e isolamento por usuário autenticado.

Dessa forma, a metodologia adotada combinou pesquisa aplicada, abordagem quantitativa e qualitativa com natureza exploratória, desenvolvimento iterativo e

incremental, modelagem de requisitos, implementação modular e validação prática da aplicação. Essa combinação permitiu que o Core 4 ERP fosse construído de maneira progressiva, acompanhando a complexidade do domínio financeiro pessoal e possibilitando que cada módulo fosse integrado ao sistema de forma coerente. O modelo escolhido mostrou-se adequado porque permitiu transformar gradualmente o problema de dispersão das informações financeiras em uma solução computacional estruturada, segura e orientada à tomada de decisão.

4.1 Tecnologias Utilizadas

A definição das tecnologias utilizadas decorreu das decisões metodológicas e arquiteturais adotadas no projeto. Como o sistema foi desenvolvido de forma incremental e organizado em módulos financeiros interdependentes, tornou-se necessário selecionar ferramentas que favorecessem separação de responsabilidades, reuso de componentes, segurança, persistência relacional, integração por API e monitoramento operacional. Assim, as tecnologias não foram escolhidas apenas por disponibilidade técnica, mas por sua adequação ao tipo de aplicação proposta: um sistema web de gestão financeira pessoal, com frontend desacoplado, backend estruturado em camadas, banco de dados relacional, autenticação segura, geração de relatórios e observabilidade.

Com base nessas decisões, as tecnologias utilizadas no desenvolvimento do Core 4 ERP foram organizadas conforme as camadas e responsabilidades da aplicação:

- Frontend: React, Vite, JavaScript, React Router, Tailwind CSS, Lucide-React, Recharts e componentes próprios.
- Backend: Java, Spring Boot, Spring Web, Spring Security, Spring Data JPA, Bean Validation, JWT, Spring AI, Bucket4j para rate limiting, Caffeine *Cache*, Spring Boot Actuator, Prometheus para monitoramento, Spring Mail para recuperação e redefinição de senha, OpenAPI/Swagger, Grafana, Loki, Promtail e Gradle.
- Banco de dados: PostgreSQL com tabelas relacionais, chaves estrangeiras, constraints e *migrations* Flyway.
- Manipulação de arquivos e relatórios: Apache POI para geração de planilhas Excel, jsPDF e jsPDF-AutoTable para geração de relatórios em PDF no frontend.

- Ferramentas de desenvolvimento e apoio: Node.js, Gradle, Swagger UI, variáveis de configuração por ambiente e estrutura local de execução para backend, frontend e banco de dados.

4.2 Etapas de Desenvolvimento

O desenvolvimento do Core 4 ERP foi organizado em etapas progressivas, acompanhando a evolução dos módulos, das regras de negócio e da compreensão do problema investigado. A definição dos requisitos não ocorreu de forma isolada, mas a partir de pesquisa exploratória sobre gestão financeira pessoal, análise de sistemas ERP e ferramentas financeiras existentes, benchmarking de funcionalidades recorrentes em soluções de mercado, observação de necessidades práticas de usuários e validação do próprio comportamento da aplicação durante o desenvolvimento. Essa combinação permitiu transformar o problema de dispersão dos registros financeiros em funcionalidades concretas, como dashboard, relatórios, conciliação OFX, calendário financeiro, controle de cartões, investimentos, assinaturas e assistente de IA.

A primeira etapa consistiu no levantamento dos módulos necessários para a gestão financeira pessoal, considerando funcionalidades como autenticação, categorias, parceiros, contas financeiras, contas correntes, cartões de crédito, investimentos, assinaturas recorrentes, conciliação bancária OFX, relatórios, calendário financeiro, dashboard e assistente de IA. Esse levantamento foi orientado pelo problema de pesquisa e pelo entendimento de que o usuário precisava concentrar, em um único ambiente, informações normalmente distribuídas em bancos, planilhas, faturas, anotações e aplicativos desconectados.

Na segunda etapa, foi realizada a análise do domínio financeiro e a definição dos principais requisitos funcionais. Essa análise considerou fluxos de controle de contas a pagar e receber, baixa e estorno, transferências, fechamento de faturas, recorrência de assinaturas, conciliação de movimentações bancárias e visualização consolidada das informações. Também foram observados padrões de sistemas de gestão e ERPs de mercado, especialmente quanto à integração entre módulos, rastreabilidade das operações e uso de relatórios gerenciais.

A terceira etapa envolveu a modelagem das entidades Java e do *schema* relacional PostgreSQL, incluindo a definição de tabelas, relacionamentos, chaves estrangeiras,

constraints e migrations Flyway. Essa etapa foi necessária para garantir que as entidades do domínio financeiro, como usuários, contas, contas baixadas, contas correntes, cartões, faturas, investimentos, assinaturas, conciliações e notificações, fossem representadas de maneira consistente no banco de dados.

A quarta etapa contemplou a implementação do backend com controllers, services, repositories, DTOs, mecanismos de segurança, validações e regras de negócio. Nessa etapa foram implementados fluxos como autenticação, isolamento dos dados por usuário, baixa e estorno de contas, fechamento de faturas, transferência entre contas correntes, geração de notificações, conciliação bancária, relatórios e integração com assistente de IA.

A quinta etapa correspondeu à implementação do frontend, com rotas protegidas, layout autenticado, telas por módulo, componentes reutilizáveis, integração HTTP com a API e recursos de visualização gráfica. O objetivo foi permitir que as funcionalidades do backend fossem apresentadas ao usuário de forma compreensível, com organização visual adequada e fluxos de interação coerentes com as tarefas financeiras realizadas.

A sexta etapa compreendeu a implementação de recursos complementares, como exportação de relatórios em Excel, geração de PDF no frontend, observabilidade com Prometheus e Grafana, monitoramento de logs, recuperação de senha por e-mail, rate limiting e bloqueio temporário de contas após tentativas malsucedidas de login.

A sétima etapa envolveu a construção dos diagramas técnicos do sistema, contemplando arquitetura, infraestrutura, entidades, casos de uso, fluxos financeiros e modelagem relacional. Esses artefatos serviram como documentação de apoio para demonstrar a relação entre requisitos, estrutura do sistema e implementação.

Por fim, realizou-se a validação funcional dos principais fluxos do sistema, a análise das rotas REST, a verificação da coerência entre entidades, serviços e interface, a aplicação de pesquisa de satisfação com usuários e o acompanhamento de métricas da aplicação durante o uso. Essa etapa permitiu avaliar tanto a percepção dos usuários quanto o comportamento operacional da aplicação.

4.3 Validação e Testes

A validação do Core 4 ERP foi planejada para verificar se os principais requisitos definidos ao longo do desenvolvimento estavam refletidos no funcionamento da aplicação

e se os usuários conseguiram compreender e utilizar os módulos implementados. Segundo Pressman e Maxim (2016), a validação busca demonstrar que o software atende às necessidades do usuário, enquanto a verificação observa se o produto foi construído de acordo com as especificações estabelecidas. Nesse sentido, a avaliação realizada neste trabalho combinou verificação funcional dos fluxos implementados, uso exploratório da aplicação, pesquisa de satisfação com usuários e monitoramento técnico por métricas operacionais.

O projeto possui estrutura de testes automatizados no backend, porém a cobertura atual ainda se limita ao teste básico de carregamento do contexto da aplicação. Por esse motivo, a validação descrita neste trabalho concentrou-se principalmente em testes funcionais manuais e exploratórios, execução dos principais fluxos de negócio, conferência das rotas REST e análise da coerência entre interface, services, repositories, entidades e banco de dados. Essa escolha é apresentada como limitação metodológica e também como direcionamento para evolução futura da qualidade do sistema.

Os testes funcionais contemplaram fluxos como autenticação, cadastro e consulta de entidades, baixa e estorno de contas, transferências entre contas correntes, lançamentos de cartão, fechamento de faturas, geração de assinaturas recorrentes, importação OFX, emissão de relatórios, utilização do calendário financeiro e uso do assistente de IA. Para cada fluxo, os critérios de aceite foram derivados dos requisitos funcionais e das regras de negócio implementadas, considerando resultados esperados como atualização correta de status, persistência dos registros, alteração de saldos, geração de notificações, exportação de relatórios e isolamento dos dados por usuário autenticado.

Embora não tenham sido implementados testes unitários específicos para todas as regras financeiras, eles seriam recomendados para serviços como baixa, estorno, transferência, fechamento de fatura, conciliação bancária, geração de notificações e recorrência de assinaturas. No backend Java, esses testes poderiam ser estruturados com JUnit 5, framework voltado à escrita e execução de testes automatizados em aplicações Java (JUNIT, 2026). Já os testes de integração poderiam validar a comunicação entre controllers, services, repositories, banco de dados e segurança, utilizando os recursos de teste disponibilizados pelo Spring Boot para carregamento do contexto, simulação de requisições HTTP e verificação de integrações com a camada de persistência (SPRING BOOT, 2026).

Também seriam cabíveis testes de aceitação para validar o sistema a partir da perspectiva do usuário final. O ISTQB (2024) destaca que testes de aceitação têm como foco demonstrar a prontidão do sistema para uso e verificar se ele atende às necessidades estabelecidas. No contexto do Core 4 ERP, esses testes poderiam ser definidos a partir de cenários como: cadastrar uma conta e realizar sua baixa; importar um arquivo OFX e concluir uma conciliação; fechar uma fatura e gerar conta a pagar; consultar o dashboard após registrar movimentações; exportar relatório financeiro; e redefinir senha por e-mail. Esses cenários conectam diretamente os requisitos aos resultados esperados pelo usuário.

Como estratégia complementar, foi realizada uma sessão de uso com usuários que não participaram diretamente do desenvolvimento do sistema, seguida da aplicação de uma pesquisa de satisfação. Essa validação permitiu coletar percepções sobre interface, desempenho, relatórios, conciliação OFX, experiência mobile, funcionalidades mais valorizadas e probabilidade de recomendação do sistema. Durante a sessão de uso, também foram monitoradas métricas da aplicação por meio de Prometheus e Grafana, incluindo taxa de erros HTTP 5xx, latência P95 por endpoint, taxa de requisições por segundo e uso de *heap* da JVM. Dessa forma, a validação combinou evidências funcionais, percepção de uso e indicadores técnicos de execução.

4.3.1 validação e monitoramento da aplicação

A validação do Core 4 ERP foi realizada com o objetivo de avaliar a percepção de usuários em relação ao uso do sistema e observar o comportamento operacional da aplicação durante a execução dos principais módulos. Para isso, foram utilizadas duas estratégias complementares: aplicação de uma pesquisa de satisfação com usuários e monitoramento técnico da aplicação por meio de métricas coletadas em tempo de execução.

A pesquisa de satisfação foi aplicada por meio de formulário online, utilizando o Google Forms, após uma sessão de uso do sistema. A realização de sessões de uso com usuários reais constitui uma estratégia reconhecida de avaliação de usabilidade, permitindo identificar aspectos de facilidade de aprendizado, eficiência de uso e satisfação (NIELSEN, 1994). Participaram 34 respondentes, selecionados por conveniência, incluindo usuários com perfil de uso pessoal, gestores financeiros, contador/analista e administrador do sistema. Os participantes utilizaram a aplicação em um contexto de validação funcional, explorando módulos como dashboard, relatórios,

lançamentos, cartões de crédito, conciliação OFX, investimentos, assinaturas, contas correntes e calendário financeiro.

O instrumento de coleta contemplou questões objetivas e abertas, incluindo perfil de uso, tempo de utilização, módulo mais acessado, probabilidade de recomendação do sistema por meio da métrica NPS, Net Promoter Score, avaliação de aspectos específicos em escala de 1 a 5 e comentários livres. Conforme Gil (2022), a amostragem por conveniência é adequada em pesquisas de caráter exploratório, cujo objetivo é obter evidências iniciais sobre o fenômeno investigado, e não generalizações estatísticas. Nesse sentido, os resultados obtidos oferecem evidências relevantes sobre a aceitação inicial do sistema, seus pontos fortes e oportunidades de melhoria.

4.3.2 Pesquisa de Satisfação com Usuários

A pesquisa contou com 34 respostas e teve caráter exploratório. O objetivo foi identificar a percepção dos usuários quanto à utilidade, usabilidade e desempenho do Core 4 ERP durante a utilização dos principais módulos implementados. Os dados foram analisados de forma agregada, sem identificação individual dos participantes, considerando a distribuição dos perfis, os módulos mais utilizados, o resultado do NPS, as médias atribuídas aos aspectos avaliados e as funcionalidades mais valorizadas.

A fim de assegurar a rastreabilidade dos dados coletados e evidenciar a efetiva participação dos respondentes, apresenta-se na Tabela 1 a relação dos 34 participantes da pesquisa de satisfação, identificados de forma anonimizada. Para cada participante são indicados o perfil de uso declarado, o tempo de utilização do Core 4 ERP, o módulo explorado com maior frequência durante a sessão de validação e a nota atribuída na escala NPS (0 a 10), com a respectiva classificação como promotor (notas 9–10), neutro (notas 7–8) ou detrator (notas 0–6). Os dados foram coletados exclusivamente por meio do formulário online disponibilizado via Google Forms, no mesmo período de aplicação da sessão de uso descrita na seção 4.3.

Tabela 1 – Relação de participantes da pesquisa de satisfação

Participante	Perfil de Uso	Tempo de Uso	Módulo Principal	NPS	Classificação
P01	Uso pessoal/ finanças pessoais	Menos de 1 mês	Dashboard	8	Neutro

Participante	Perfil de Uso	Tempo de Uso	Módulo Principal	NPS	Classificação
P02	Uso pessoal/ finanças pessoais	Menos de 1 mês	Dashboard	10	Promotor
P03	Gestor financeiro	Menos de 1 mês	Dashboard	9	Promotor
P04	Uso pessoal/ finanças pessoais	Menos de 1 mês	Relatórios	8	Neutro
P05	Uso pessoal/ finanças pessoais	Menos de 1 mês	Assinaturas	10	Promotor
P06	Gestor financeiro	Menos de 1 mês	Lançamentos	10	Promotor
P07	Uso pessoal/ finanças pessoais	Menos de 1 mês	Lançamentos	10	Promotor
P08	Uso pessoal/ finanças pessoais	Menos de 1 mês	Dashboard	10	Promotor
P09	Uso pessoal/ finanças pessoais	Menos de 1 mês	Lançamentos	10	Promotor
P10	Uso pessoal/ finanças pessoais	Menos de 1 mês	Dashboard	8	Neutro
P11	Contador / Analista	Menos de 1 mês	Lançamentos	10	Promotor
P12	Uso pessoal/ finanças pessoais	Menos de 1 mês	Relatórios	6	Detrator
P13	Uso pessoal/ finanças pessoais	Menos de 1 mês	Dashboard	6	Detrator
P14	Uso pessoal/ finanças pessoais	Menos de 1 mês	Contas correntes	2	Detrator
P15	Gestor financeiro	Mais de 6 meses	Investimentos	5	Detrator
P16	Administrador do sistema	3 a 6 meses	Dashboard	7	Neutro

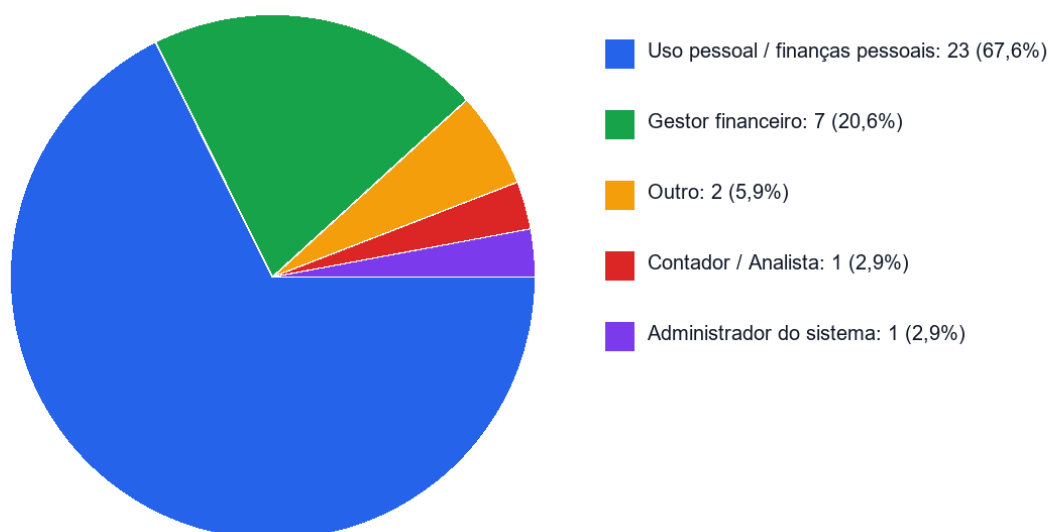
Participante	Perfil de Uso	Tempo de Uso	Módulo Principal	NPS	Classificação
P17	Uso pessoal/ finanças pessoais	1 a 3 meses	Dashboard	10	Promotor
P18	Uso pessoal/ finanças pessoais	Menos de 1 mês	Cartões de crédito	10	Promotor
P19	Uso pessoal/ finanças pessoais	1 a 3 meses	Conciliação OFX	9	Promotor
P20	Uso pessoal/ finanças pessoais	Menos de 1 mês	Relatórios	10	Promotor
P21	Gestor financeiro	Menos de 1 mês	Conciliação OFX	9	Promotor
P22	Gestor financeiro	Mais de 6 meses	Relatórios	10	Promotor
P23	Outro	Menos de 1 mês	Calendário	5	Detrator
P24	Uso pessoal/ finanças pessoais	Menos de 1 mês	Relatórios	10	Promotor
P25	Uso pessoal/ finanças pessoais	1 a 3 meses	Relatórios	10	Promotor
P26	Gestor financeiro	Menos de 1 mês	Conciliação OFX	10	Promotor
P27	Uso pessoal/ finanças pessoais	1 a 3 meses	Relatórios	9	Promotor
P28	Gestor financeiro	3 a 6 meses	Relatórios	10	Promotor
P29	Uso pessoal/ finanças pessoais	Menos de 1 mês	Dashboard	7	Neutro
P30	Uso pessoal/ finanças pessoais	Menos de 1 mês	Relatórios	10	Promotor
P31	Uso pessoal/ finanças pessoais	Menos de 1 mês	Cartões de crédito	8	Neutro

Participante	Perfil de Uso	Tempo de Uso	Módulo Principal	NPS	Classificação
P32	Outro	Menos de 1 mês	Dashboard	10	Promotor
P33	Uso pessoal/ finanças pessoais	Menos de 1 mês	Cartões de crédito	9	Promotor
P34	Uso pessoal/ finanças pessoais	3 a 6 meses	Investimentos	9	Promotor

Fonte: Elaborado pelo autor com base nos dados coletados via Google Forms.

Gráfico 1 - Perfil dos participantes da pesquisa de satisfação

Perfil dos participantes

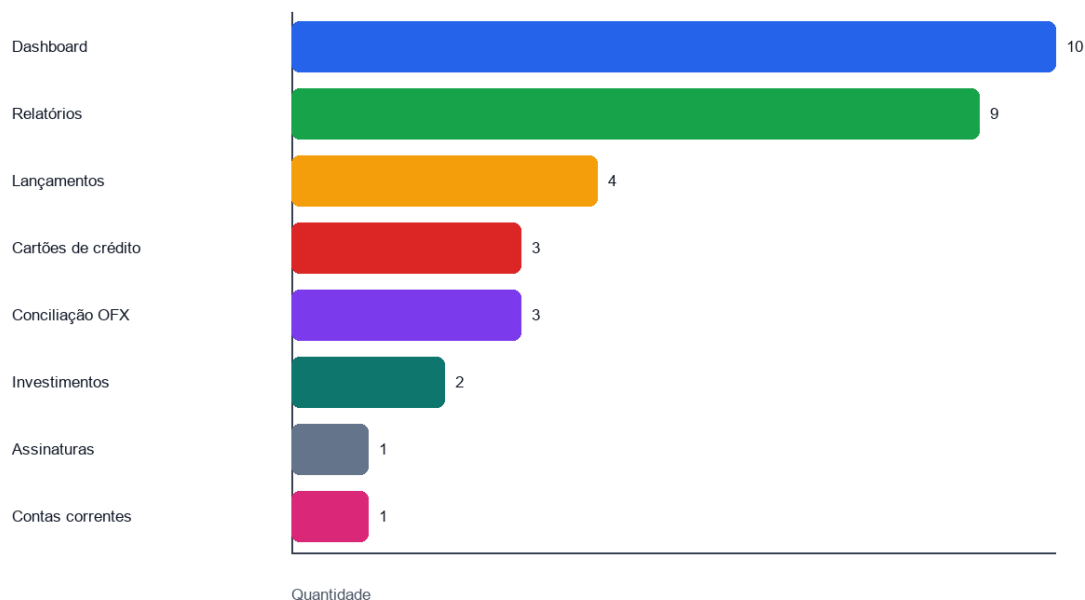


Fonte: Elaborado pelo autor com base nos dados da pesquisa.

A maior parte dos respondentes declarou utilizar o sistema para fins de organização financeira pessoal, representando 67,6% da amostra. Esse resultado é coerente com o público-alvo principal do Core 4 ERP, que é a gestão financeira pessoal. A presença de gestores financeiros, contador/analista e administrador do sistema ampliou a diversidade da avaliação, permitindo observar a percepção de usuários com diferentes níveis de familiaridade com rotinas financeiras e sistemas de informação.

Gráfico 2 - Módulos utilizados com maior frequência

Módulos utilizados com maior frequência



Fonte: Elaborado pelo autor com base nos dados da pesquisa.

Os módulos mais utilizados foram Dashboard e Relatórios, que juntos concentraram 55,9% das respostas. Esse resultado indica que os usuários atribuíram maior relevância às funcionalidades de visualização consolidada e análise financeira. Também houve uso de módulos operacionais, como lançamentos, cartões de crédito, conciliação OFX, investimentos, assinaturas, contas correntes e calendário, demonstrando que a validação contemplou diferentes áreas funcionais do sistema.

Tabela 2 - Resultado da avaliação NPS

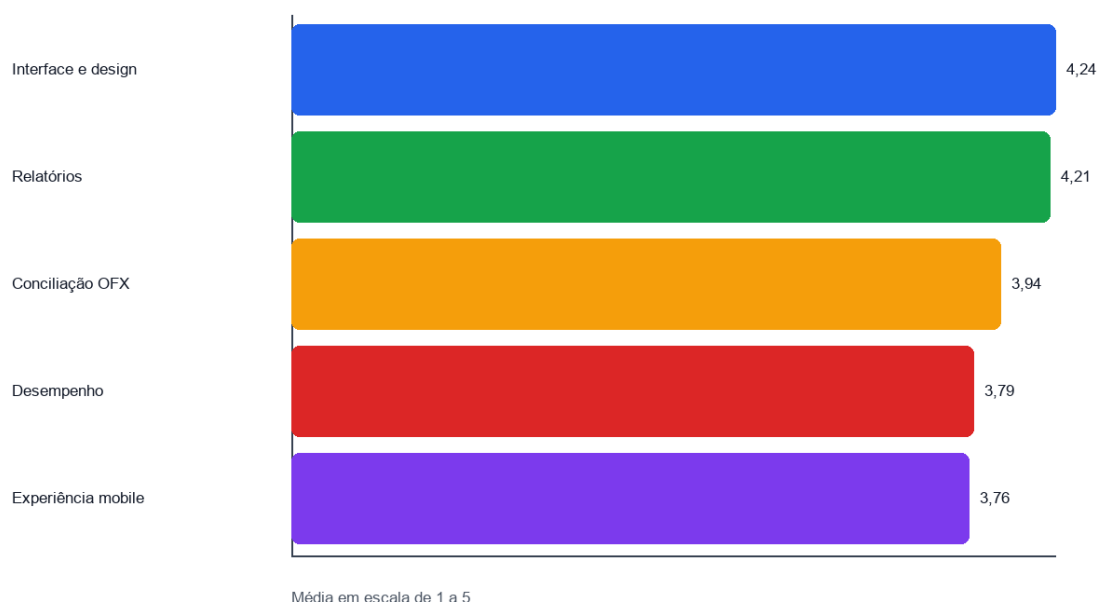
Classificação	Critério	Quantidade	Percentual
Promotores	Notas 9 e 10	23	67,6%
Neutros	Notas 7 e 8	6	17,6%
Detratores	Notas 0 a 6	5	14,7%
NPS final		52,9	

Fonte: Elaborado pelo autor com base nos dados da pesquisa.

O NPS final de 52,9 indica uma percepção geral positiva em relação ao sistema, com predominância de usuários promotores. Esse resultado sugere que, no contexto avaliado, o Core 4 ERP apresentou boa aceitação e potencial de recomendação. Entretanto, a existência de usuários neutros e detratores evidencia que ainda há pontos de melhoria, especialmente nos aspectos relacionados ao desempenho, à experiência mobile e a ajustes de usabilidade relatados nas respostas abertas.

Gráfico 3 - Avaliação média dos aspectos do sistema

Avaliação média dos aspectos do sistema



Fonte: Elaborado pelo autor com base nos dados da pesquisa.

A análise das médias mostra que os aspectos mais bem avaliados foram interface e *design*, com média 4,24, e relatórios, com média 4,21. Esses resultados indicam boa aceitação da organização visual da aplicação e das funcionalidades de análise financeira. A conciliação OFX apresentou média 3,94, resultado positivo, mas que também sugere possibilidade de aprimoramento na clareza do fluxo de importação, sugestão e finalização das conciliações.

Os menores resultados foram observados em desempenho, com média 3,79, e experiência mobile, com média 3,76. Esses dados indicam que, embora a aplicação tenha

sido funcionalmente aceita, existem oportunidades de melhoria relacionadas à otimização de carregamento, responsividade, adaptação de componentes para telas menores e redução de possíveis travamentos ou lentidão em determinadas consultas. Essa percepção também se conecta às respostas abertas, nas quais foram mencionadas dificuldades pontuais de carregamento e uso em dispositivos móveis.

Tabela 3 - Funcionalidades mais valorizadas pelos usuários

Funcionalidade	Quantidade
Dashboard completo	22
Design moderno	20
Segurança dos dados	18
Relatórios em Excel e PDF	14
Conciliação bancária automática	14
Chat IA integrado	11
Calendário financeiro	11
Gestão de cartões e faturas	10
Velocidade da interface	10

Fonte: Elaborado pelo autor com base nos dados da pesquisa.

As funcionalidades mais valorizadas foram dashboard completo, design moderno e segurança dos dados, demonstrando que os usuários perceberam valor tanto na apresentação visual quanto na confiabilidade do sistema. Também se destacaram recursos mais específicos do projeto, como relatórios em Excel e PDF, conciliação bancária automática, chat de IA integrado, calendário financeiro e gestão de cartões e faturas. Esses resultados reforçam a relevância dos diferenciais implementados no Core 4 ERP e indicam que os módulos avançados foram percebidos como úteis pelos participantes.

De modo geral, os resultados da pesquisa indicam boa aceitação inicial do Core 4 ERP pelos usuários participantes. Entretanto, por se tratar de uma amostra selecionada por conveniência e de caráter exploratório, os dados não devem ser interpretados como generalização estatística, mas como evidências de percepção de uso em um cenário controlado de validação. Os resultados apontam como principais pontos fortes a visualização consolidada, os relatórios, o design, a segurança percebida e os recursos avançados, como conciliação OFX e assistente de IA. Como oportunidades de melhoria, destacam-se a otimização de desempenho, o aprimoramento da experiência mobile e a correção de ajustes pontuais de usabilidade.

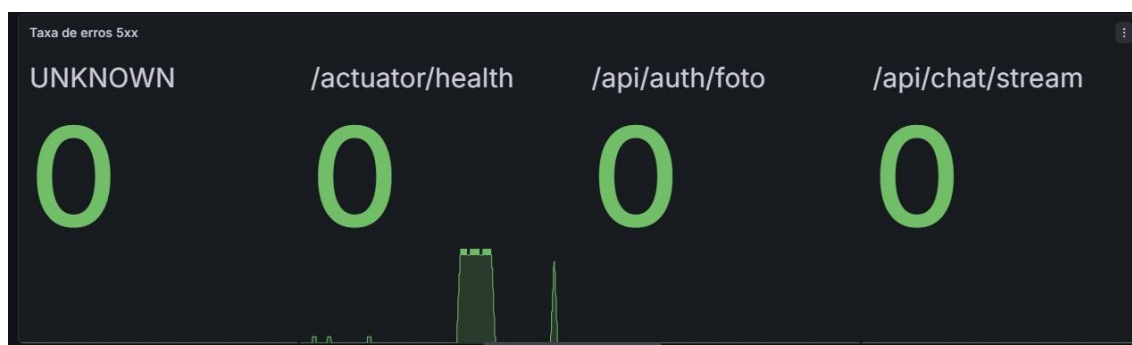
Os dados de monitoramento técnico complementam os resultados da pesquisa de satisfação. Enquanto o formulário indicou que desempenho e experiência mobile foram os aspectos com menores médias, as métricas do grafana permitiram observar o comportamento da aplicação durante o uso, especialmente em relação à latência, volume de requisições, erros HTTP 5xx e consumo de memória. Dessa forma, a análise conjunta entre percepção dos usuários e métricas operacionais contribui para identificar pontos de melhoria com maior precisão.

Durante a sessão de testes com usuários, a aplicação foi acompanhada por meio de dashboards no grafana, com métricas coletadas pelo prometheus a partir dos endpoints expostos pelo spring boot actuator. essa estrutura de observabilidade permitiu monitorar o comportamento operacional do backend durante a utilização do sistema, considerando indicadores como taxa de erros http 5xx, latência por endpoint, taxa de requisições por segundo e uso de memória heap da JVM.

As métricas observadas indicaram estabilidade da aplicação no cenário testado, sem ocorrência de erros internos 5xx nos endpoints destacados e com consumo de memória controlado durante a execução das funcionalidades avaliadas. Os picos pontuais de latência e requisições refletem os momentos de maior interação dos usuários com módulos específicos, como autenticação, consultas, relatórios, conciliação e assistente de IA, não caracterizando indisponibilidade do sistema.

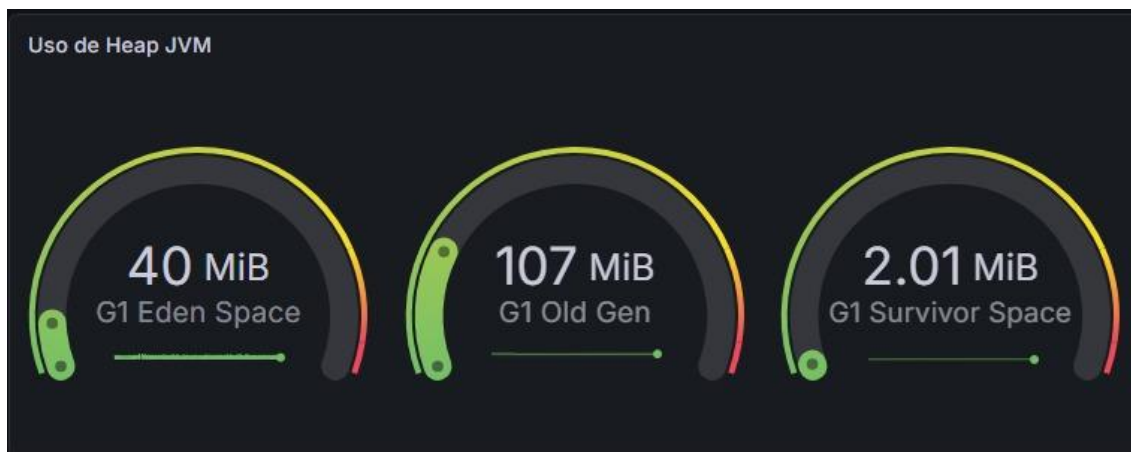
Dessa forma, o uso de prometheus e grafana contribuiu para complementar a validação funcional com evidências técnicas de monitoramento, permitindo observar o comportamento da aplicação em tempo de execução e reforçando a importância da observabilidade em sistemas web que manipulam informações financeiras.

Figura 1 - Taxa de erros 5xx por endpoint durante os testes com usuários.



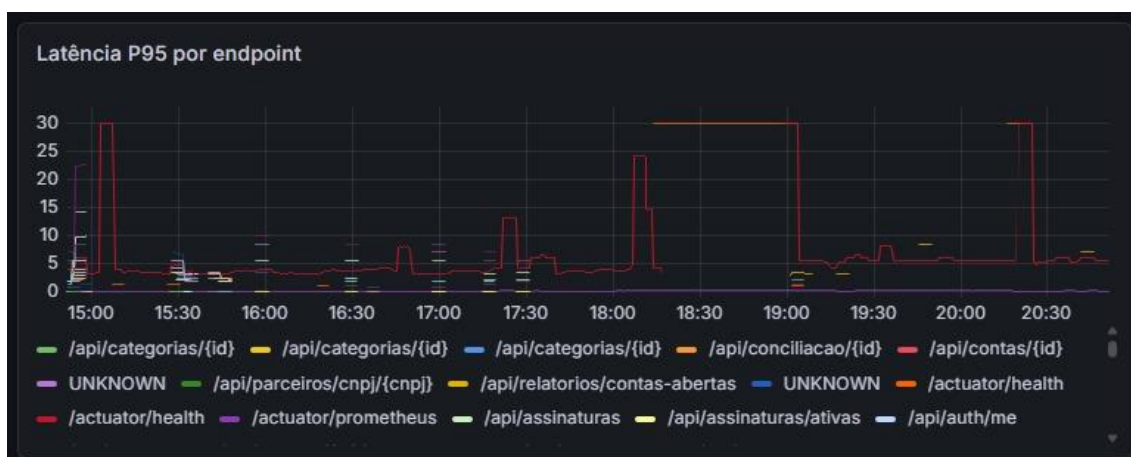
Fonte: Elaborada pelo autor

Figura 2 - Uso de memória heap da JVM durante a execução da aplicação.



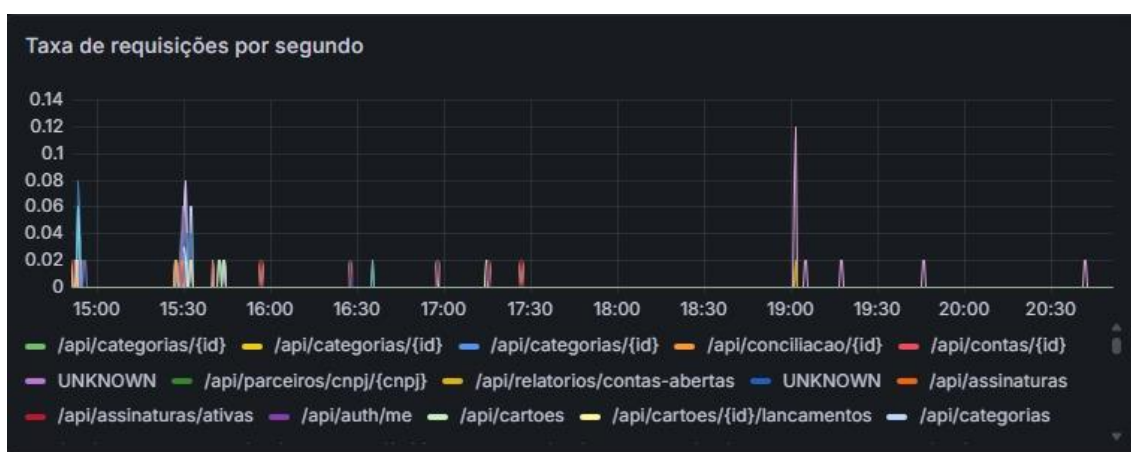
Fonte: Elaborada pelo autor

Figura 3 - Latência P95 por endpoint durante os testes com usuários.



Fonte: Elaborada pelo autor

Figura 4 - Taxa de requisições por segundo durante os testes com usuários.



Fonte: Elaborada pelo autor

A análise conjunta dessas métricas demonstra que, durante o período observado, o sistema respondeu adequadamente às interações realizadas pelos usuários, mantendo disponibilidade dos principais endpoints, ausência de erros internos recorrentes e consumo de recursos compatível com o ambiente de execução utilizado nos testes.

Embora não tenham sido observados erros internos 5xx recorrentes nos endpoints destacados, os picos de latência registrados indicam pontos que podem ser investigados em ciclos futuros de otimização. Essa análise é coerente com a avaliação dos usuários, que atribuíram médias menores ao desempenho e à experiência mobile. Assim, os resultados da validação sugerem que o sistema apresenta estabilidade funcional no cenário testado, mas ainda pode evoluir em aspectos de performance, responsividade e refinamento da experiência de uso.

4.3.3 Métricas de Engenharia de Software

A avaliação de um sistema de software não se limita à verificação funcional dos requisitos implementados. A mensuração de indicadores quantitativos da base de código constitui prática relevante da Engenharia de Software, pois permite evidenciar atributos como manutenibilidade, complexidade, organização arquitetural e cobertura de testes. Pressman e Maxim (2016) destacam que métricas de software fornecem uma base objetiva para a avaliação da qualidade do produto e do processo de desenvolvimento, possibilitando comparações, diagnósticos e decisões técnicas fundamentadas. Nesse sentido, a presente seção reúne os indicadores quantitativos levantados sobre o Core 4 ERP, abrangendo linhas de código (LOC), número de classes e tipos declarados, endpoints REST, tabelas do banco de dados, complexidade ciclomática e situação dos testes automatizados.

Os indicadores foram coletados a partir da análise estática do código-fonte da aplicação na branch principal (main), com data de referência em 10 de junho de 2026. O escopo de medição contemplou o backend Java com Spring Boot, o frontend React com Vite e as migrations de banco de dados gerenciadas pelo Flyway, excluindo dependências de terceiros, como `node_modules` e bibliotecas Gradle. A Tabela 4 apresenta o resumo executivo com os principais indicadores obtidos.

Tabela 4 – Resumo executivo das métricas de engenharia de software

Indicador	Valor
-----------	-------

Linhas de código (LOC) — total do projeto	17.603
Linhas de código — backend (Java)	8.315
Linhas de código — frontend (React/JSX)	8.668
Linhas de código — banco de dados (SQL/Flyway)	620
Arquivos-fonte Java	154
Arquivos-fonte frontend (.jsx/.js)	67
Tipos Java declarados (classes, interfaces, enums, records)	162
Endpoints REST	95
Tabelas no banco de dados	16
Migrations Flyway	11
Complexidade ciclomática total (estimada)	~948
Complexidade ciclomática média por método	~2,2
Classes de teste automatizado	1
Cobertura de testes (lógica de negócio)	~0%

Fonte: Elaborado pelo autor com base nos dados do projeto (2026).

O projeto totaliza 17.603 linhas de código efetivas, distribuídas entre 8.315 no backend Java, 8.668 no frontend React e 620 no banco de dados em SQL. Essa distribuição equilibrada entre as duas camadas principais da aplicação reflete a natureza full-stack do desenvolvimento. Kaner, Falk e Nguyen (1993) observam que o volume de código não é, por si só, indicador de qualidade, mas serve como referência para dimensionar o escopo da solução e contextualizar outros indicadores, como complexidade e cobertura de testes.

O backend contém 154 arquivos-fonte Java, totalizando 162 tipos declarados quando considerados também os tipos aninhados. A Tabela 5 detalha a distribuição por categoria de declaração, evidenciando o uso extensivo de records para a implementação

de DTOs imutáveis — padrão que favorece a separação entre o contrato da API e as entidades de persistência.

Tabela 5 – Distribuição de tipos declarados no backend Java

Tipo de declaração	Quantidade
class	85
record (DTOs imutáveis)	50
interface	18
enum	9
Total	162

Fonte: Elaborado pelo autor com base nos dados do projeto (2026).

A Tabela 6 apresenta a distribuição dos arquivos por camada arquitetural, segundo o padrão MVC adotado. A concentração de 43 arquivos na camada de DTOs e 27 na de serviços evidencia a separação de responsabilidades implementada, com a lógica de negócio isolada na camada de services e o contrato da API expresso por meio de objetos de transferência dedicados.

Tabela 6 – Distribuição de arquivos por camada arquitetural (MVC)

Camada	Arquivos	Anotação Spring principal
Controllers (REST)	14	@RestController
Services (regra de negócio)	27	@Service
Repositories (persistência)	17	extends JpaRepository
Entities (mapeamento JPA)	16	@Entity
DTOs (objetos de transferência)	43	—
Configurações	14	@Configuration / @Component
Enums de domínio	9	—

Tratamento de exceções	3	@RestControllerAdvice
------------------------	---	-----------------------

Fonte: Elaborado pelo autor com base nos dados do projeto (2026).

A API REST do Core 4 ERP expõe 95 endpoints, distribuídos pelos principais métodos HTTP conforme apresentado na Tabela 7. A predominância de operações GET, com 43 endpoints (45,3%), reflete a natureza da aplicação, voltada à consulta e visualização de informações financeiras. Os 24 endpoints POST concentram a criação de recursos, enquanto PUT e DELETE contemplam operações de atualização completa e remoção.

Tabela 7 – Distribuição dos endpoints REST por método HTTP

Método HTTP	Quantidade	Uso predominante
GET	43	Consultas e listagens
POST	24	Criação de recursos
PUT	13	Atualização completa
DELETE	12	Remoção de recursos
PATCH	3	Atualização parcial
Total	95	—

Fonte: Elaborado pelo autor com base nos dados do projeto (2026).

O esquema relacional do sistema é composto por 16 tabelas, estruturadas por meio de 11 migrations Flyway versionadas. A Tabela 8 apresenta a relação completa das tabelas e seus respectivos domínios funcionais. A adoção da estratégia ddl-auto=validate garante que o DDL seja gerenciado exclusivamente pelo Flyway, nunca pelo Hibernate, o que assegura maior controle sobre a evolução do schema e evita alterações não rastreadas na estrutura do banco.

Tabela 8 – Tabelas do banco de dados e respectivos domínios

#	Tabela	Domínio
1	tb_usuario	Usuários (âncora de multitenancy por usuário autenticado)

2	tb_categoria	Categorias de receita e despesa
3	tb_parceiro	Fornecedores e clientes
4	tb_conta	Contas a pagar e a receber
5	tb_conta_baixada	Histórico de baixas financeiras
6	tb_conta_corrente	Contas bancárias correntes
7	tb_cartao_credito	Cartões de crédito
8	tb_fatura_cartao	Faturas de cartão
9	tb_lancamento_cartao	Lançamentos por fatura
10	tb_conta_investimento	Contas de investimento
11	tb_tipo_investimento	Tipos de investimento
12	tb_transacao_investimento	Transações de investimento
13	tb_assinatura	Assinaturas recorrentes
14	tb_conciliacao	Conciliação bancária (OFX)
15	tb_conciliacao_item	Itens de conciliação
16	tb_notificacao	Notificações

Fonte: Elaborado pelo autor com base nos dados do projeto (2026).

A complexidade ciclomática foi estimada por análise estática dos pontos de decisão presentes no código Java do backend. Conforme McCabe (1976), a complexidade ciclomática mede o número de caminhos linearmente independentes em um módulo, sendo amplamente utilizada como indicador de manutenibilidade e testabilidade do código. A Tabela 9 apresenta o detalhamento dos pontos de decisão contabilizados.

Tabela 9 – Estimativa de complexidade ciclomática (backend Java)

Métrica	Valor
Métodos analisados (aprox.)	426

Estruturas if	285
Operadores lógicos (&& /)	132
Laços for	58
Blocos catch	31
case (switch)	9
Laços while	7
Pontos de decisão (total)	522
Complexidade ciclomática total estimada	~948
Complexidade ciclomática média por método	~2,2

Fonte: Elaborado pelo autor com base nos dados do projeto (2026).

A complexidade ciclomática média de ~2,2 por método é considerada baixa pela literatura. McCabe (1976) sugere atenção a partir de complexidade 10 e classifica como risco elevado valores superiores a 20. O resultado obtido indica métodos curtos e com responsabilidades bem delimitadas, coerente com a separação em camadas adotada e com as boas práticas de design de software que recomendam métodos coesos e de propósito único. Cabe registrar que o valor representa uma estimativa por contagem textual e não substitui uma mensuração formal por ferramenta de análise de AST, como SonarQube ou JaCoCo.

No que se refere à cobertura de testes automatizados, a Tabela 10 sintetiza a situação atual do projeto. O único teste existente verifica exclusivamente se o contexto da aplicação Spring inicializa sem erros, sem exercitar regras de negócio, endpoints ou repositórios. Essa limitação é reconhecida como oportunidade de melhoria e está registrada nos trabalhos futuros do projeto.

Tabela 10 – Situação atual dos testes automatizados

Métrica	Valor
Classes de teste	1 (Core4erpApplicationTests)

Métodos de teste	1 (contextLoads)
Tipo de teste existente	Smoke test de carregamento do contexto Spring
Cobertura de lógica de negócio	~0%
Ferramenta de cobertura (JaCoCo)	Não configurada
Relação teste : produção (classes)	1 : 154 (~0,65%)

Fonte: Elaborado pelo autor com base nos dados do projeto (2026).

Apesar da cobertura de testes reduzida, o `build.gradle` já declara as dependências necessárias para evolução da suíte, incluindo `spring-boot-starter-test`, que fornece JUnit 5, Mockito, AssertJ, MockMvc e JSONPath, além de `spring-security-test`, que oferece suporte a testes de endpoints protegidos por autenticação JWT. Isso significa que a ampliação da cobertura pode ser realizada sem a necessidade de inclusão de novas bibliotecas, constituindo um caminho direto para o fortalecimento da qualidade do sistema em versões futuras.

5. DESCRIÇÃO DA ARQUITETURA DO SISTEMA E INTERFACE

5.1 Arquitetura do Sistema

A arquitetura do Core 4 ERP foi estruturada para sustentar uma aplicação web de gestão financeira pessoal composta por módulos interdependentes, regras financeiras transacionais e recursos de análise. O sistema adota uma organização em camadas, separando a apresentação, a aplicação, as regras de negócio, a persistência e o banco de dados. Essa separação favorece a manutenção, a evolução incremental e a rastreabilidade das operações, pois cada responsabilidade passa a ser tratada em uma camada específica, reduzindo acoplamentos indevidos entre interface, lógica de negócio e armazenamento.

Na camada de apresentação, o frontend desenvolvido em React concentra as telas, a navegação, os componentes visuais e a comunicação com a API. Na camada de aplicação, o backend em Spring Boot reúne os controllers rest, services, DTOs, mecanismos de segurança, validações e integrações. A camada de persistência utiliza repositories JPA para acesso ao banco de dados PostgreSQL, responsável por armazenar usuários, contas, cartões, investimentos, conciliações, relatórios e demais entidades do

domínio financeiro. O fluxo geral inicia-se no navegador do usuário; após a autenticação, o frontend aciona endpoints protegidos, o backend valida o JWT, identifica o usuário autenticado e executa as operações com base no *usuarioid*, garantindo isolamento lógico entre os registros.

Além das camadas funcionais, a arquitetura contempla recursos de observabilidade para acompanhar o comportamento da aplicação em execução. O backend expõe métricas por meio do Spring Boot Actuator, coletadas pelo Prometheus e visualizadas no Grafana. Para o acompanhamento de logs, a *stack* utiliza Loki e Promtail, permitindo agregação e consulta de eventos da aplicação. Essa estrutura é relevante porque sistemas financeiros exigem acompanhamento de disponibilidade, latência, erros e consumo de recursos, especialmente quando operações como baixa, estorno, conciliação bancária e geração de relatórios dependem de consistência entre diferentes módulos.

O Diagrama de Arquitetura do Sistema ilustra a organização geral do Core 4 ERP sob o modelo cliente-servidor, evidenciando a separação entre a camada de apresentação em React, a camada de aplicação em Spring Boot e a camada de persistência em PostgreSQL. O diagrama demonstra como as requisições partem do navegador, passam por rotas protegidas e filtros de segurança, alcançam os controllers e services do backend e, posteriormente, são persistidas por meio do Spring Data JPA. A versão completa encontra-se disponível no Apêndice A, Figura 18.

5.2 Módulos Implementados

Os módulos implementados no Core 4 ERP foram definidos com base nos requisitos levantados para centralizar as informações financeiras do usuário e reduzir a fragmentação dos registros. Em vez de funcionarem como partes isoladas, esses módulos compartilham dados por meio da mesma base relacional e executam regras de negócio integradas, permitindo que uma ação realizada em uma área do sistema produza reflexos consistentes em outras, como ocorre nas baixas, transferências, faturas, relatórios e conciliações.

O núcleo operacional do sistema é composto pelos módulos de autenticação e perfil, categorias, parceiros, contas financeiras, contas correntes, cartões de crédito, investimentos, assinaturas recorrentes, notificações, dashboard, conciliação bancária OFX, calendário financeiro, relatórios financeiros e assistente de IA. O módulo de

autenticação controla registro, login, logout, consulta do usuário autenticado, atualização de perfil e recuperação de senha. Os módulos de categorias e parceiros fornecem cadastros auxiliares para classificação e relacionamento das movimentações financeiras, incluindo consulta de CNPJ por meio da BrasilAPI.

Os módulos transacionais concentram as principais operações financeiras do sistema. Contas financeiras permitem controlar contas a pagar e a receber, parcelamentos, baixas e estornos. Contas correntes mantêm o cadastro de contas bancárias e seus saldos, além de possibilitar transferências entre contas do mesmo usuário. Cartões de crédito abrangem cadastro, lançamentos, parcelamentos e fechamento de faturas. Investimentos permitem acompanhar contas e transações de aporte, resgate ou rendimento, enquanto assinaturas recorrentes automatizam cobranças periódicas e reduzem a necessidade de registros manuais repetitivos.

Os módulos analíticos e de apoio ampliam a capacidade de interpretação dos dados. O dashboard consolida saldos, contas, cartões, investimentos e indicadores financeiros em uma visão inicial. Os relatórios permitem analisar fluxo de caixa, DRE, extrato, contas abertas, investimentos, cartões, posição financeira e assinaturas, com exportação em Excel e PDF. A conciliação bancária OFX aproxima os registros internos dos extratos bancários, permitindo importar transações, sugerir vínculos, criar contas, realizar baixas, ignorar itens e finalizar conciliações. O calendário financeiro organiza contas e assinaturas por data de vencimento, enquanto o assistente de IA apoia consultas, lançamentos e geração de relatórios.

5.3 Diagramas Estruturais e Comportamentais

Os diagramas estruturais e comportamentais foram utilizados para documentar a organização interna do Core 4 ERP e demonstrar a relação entre requisitos, arquitetura, dados e fluxos operacionais. Como esses artefatos possuem grande dimensão e elevada quantidade de elementos, sua inserção direta no corpo do trabalho prejudicaria a leitura e a visualização dos detalhes. Por esse motivo, todos os diagramas completos foram deslocados para o Apêndice A, onde estão disponibilizados em alta resolução. No corpo do texto, são apresentadas apenas as chamadas explicativas para cada diagrama, preservando a continuidade da análise sem comprometer a legibilidade.

O Diagrama de Infraestrutura complementa a visão arquitetural ao representar a comunicação entre frontend, backend, banco de dados e ferramentas de observabilidade,

como Prometheus, Grafana, Loki e Promtail. Essa visão demonstra que a aplicação foi concebida considerando não apenas funcionalidades, mas também operação, métricas, logs e acompanhamento do ambiente de execução. A versão completa encontra-se disponível no Apêndice A, Figura 19.

O Diagrama de Classes representa a estrutura estática do domínio do sistema por meio das entidades Java, seus atributos e relacionamentos. O modelo evidencia a centralidade da classe *Usuario*, o uso de entidades financeiras como *Conta*, *ContaBaixada*, *ContaCorrente*, *CartaoCredito*, *FaturaCartao*, *LancamentoCartao*, *ContaInvestimento*, *TransacaoInvestimento*, *Assinatura*, *Conciliacao* e *Notificacao*, além da herança de campos de auditoria por meio da estrutura *Auditable*. A versão completa encontra-se disponível no Apêndice A, Figura 20.

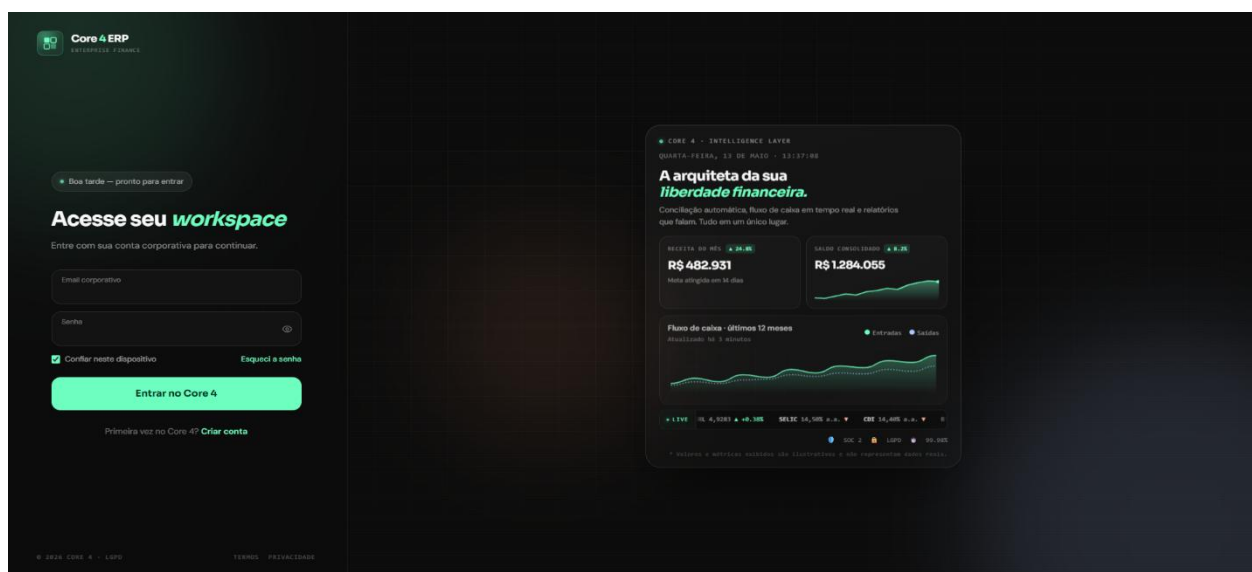
O Diagrama de Sequência descreve o fluxo dinâmico de uma operação financeira, demonstrando a sequência de interações entre interface, camada de comunicação HTTP, controller, service, repositories e banco de dados. O fluxo evidencia que a baixa ou conciliação de uma conta envolve validação do usuário autenticado, verificação da conta, cálculo de valores, atualização de status, registro da baixa e alteração do saldo da conta corrente. A versão completa encontra-se disponível no Apêndice A, Figura 21.

O Diagrama de Casos de Uso apresenta as principais interações entre o usuário e o sistema, abrangendo autenticação, gerenciamento de perfil, categorias, parceiros, contas, contas correntes, cartões, investimentos, assinaturas, conciliações, relatórios, dashboard, notificações, calendário financeiro e assistente de IA. O diagrama demonstra que o usuário atua como agente central dos fluxos operacionais e analíticos. A versão completa encontra-se disponível no Apêndice A, Figura 22.

5.4 Interface do Sistema

As telas do sistema apresentadas a seguir demonstram como os módulos implementados foram organizados na interface do Core 4 ERP. As imagens não possuem apenas função ilustrativa; elas evidenciam a aplicação prática dos requisitos funcionais, a estrutura de navegação, a disposição dos componentes e a forma como o usuário interage com cadastros, lançamentos, relatórios, calendário, dashboard e demais recursos financeiros.

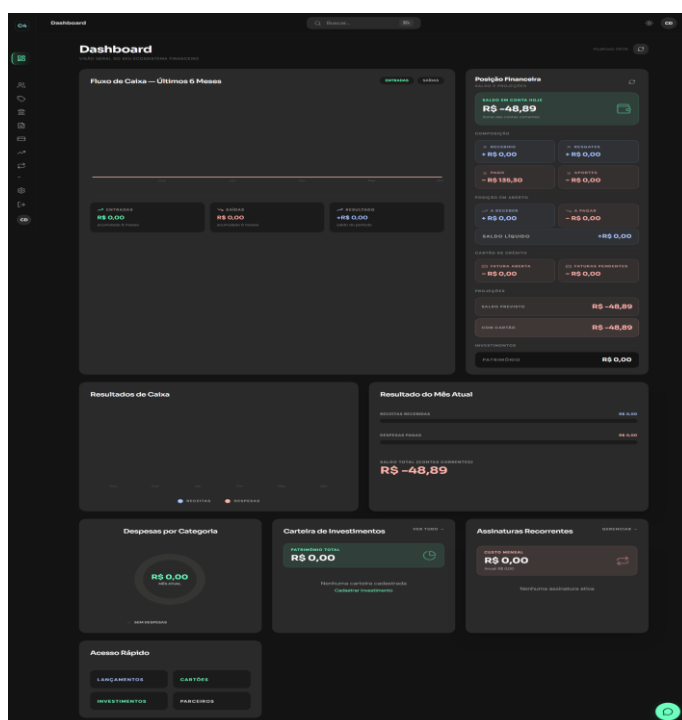
Figura 5 - Tela de Login



Fonte: Elaborada pelo autor

A Figura 6 apresenta a tela de autenticação do sistema, na qual o usuário informa suas credenciais para acessar a plataforma. Essa interface representa o ponto inicial de proteção da aplicação, pois somente após login válido o usuário passa a acessar rotas privadas e dados financeiros vinculados à sua conta.

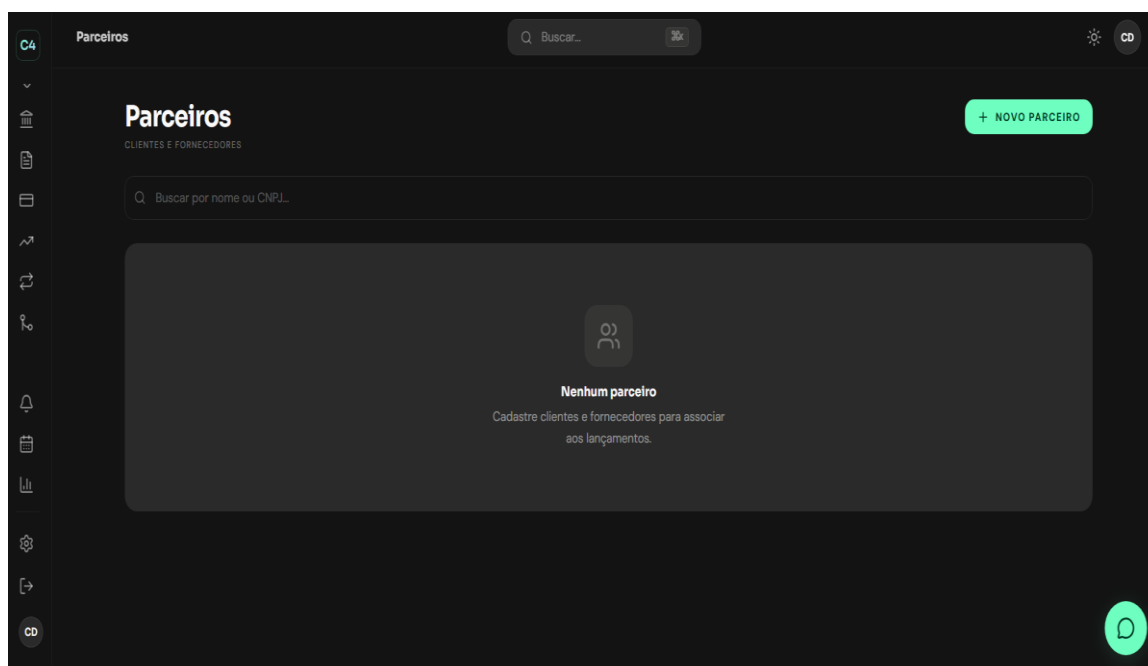
Figura 6 - Dashboard / tela inicial



Fonte: Elaborada pelo autor

A Figura 7 apresenta o dashboard, que funciona como painel inicial de análise financeira. Nessa tela, o usuário visualiza indicadores, saldos, gráficos e atalhos para módulos relevantes, permitindo uma compreensão rápida da situação financeira consolidada.

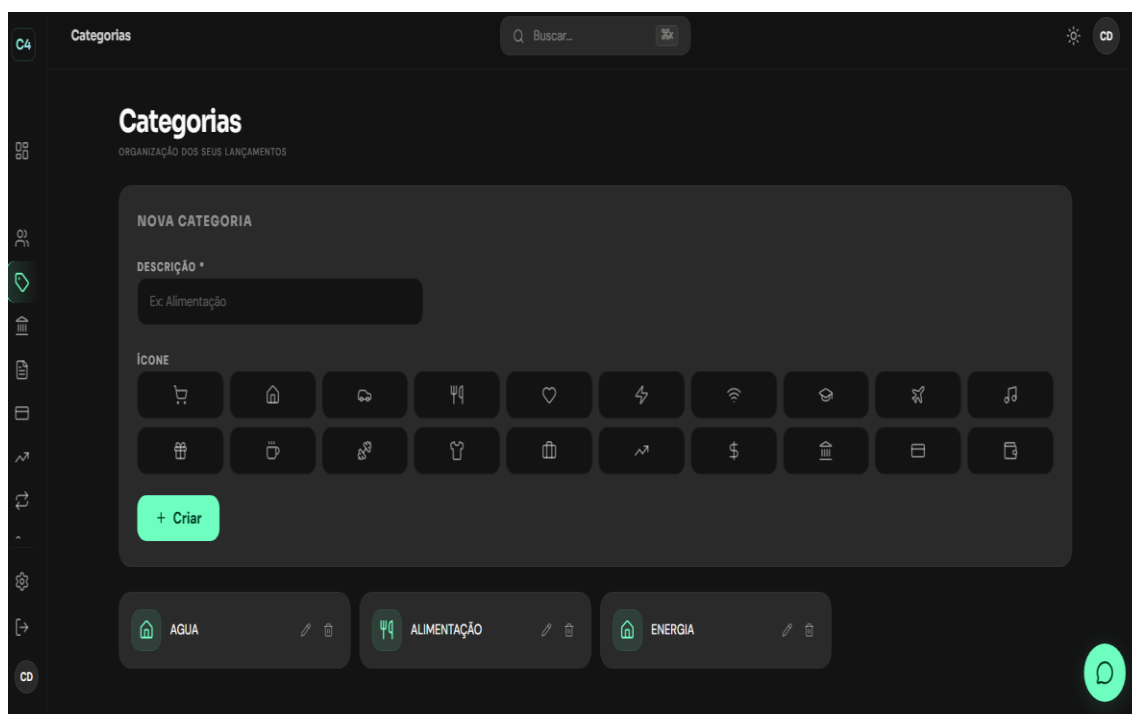
Figura 7 - Página de Parceiros



Fonte: Elaborada pelo autor

A Figura 8 apresenta a página de parceiros, destinada ao cadastro e gerenciamento de clientes, fornecedores, instituições ou demais entidades relacionadas às movimentações financeiras. Esse cadastro auxilia na rastreabilidade das contas e na organização dos lançamentos.

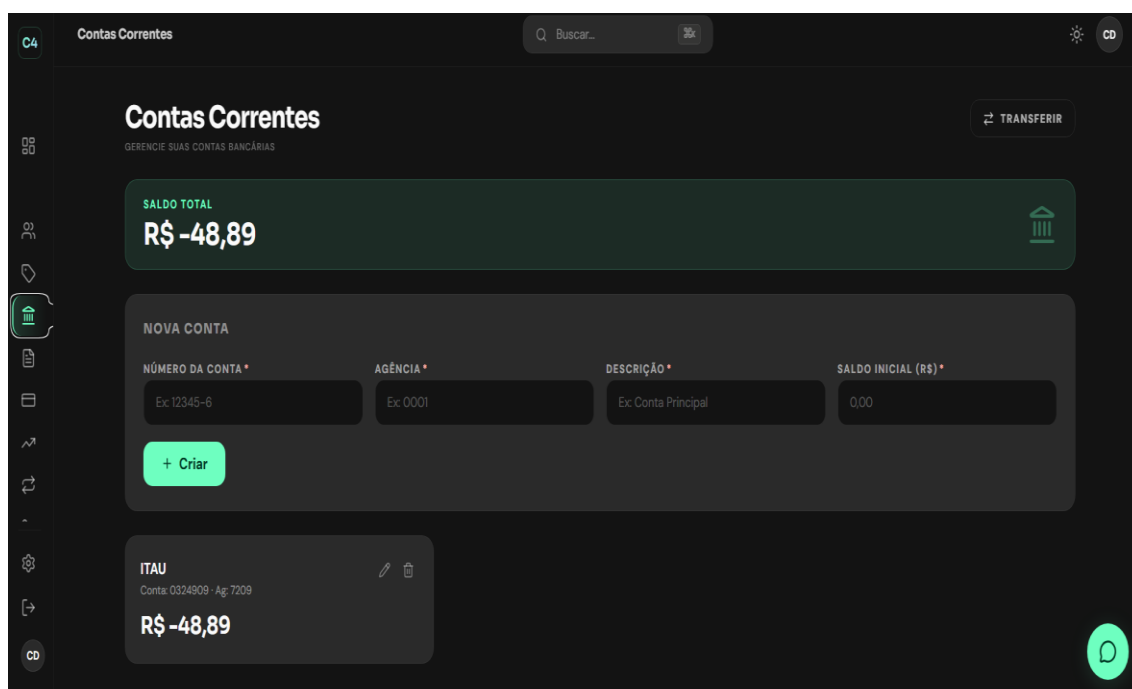
Figura 8 - Página de Categorias



Fonte: Elaborada pelo autor

A Figura 9 apresenta a tela de categorias, utilizada para estruturar grupos de receitas e despesas. A categorização é essencial para filtros, relatórios, dashboard e análise dos padrões financeiros do usuário.

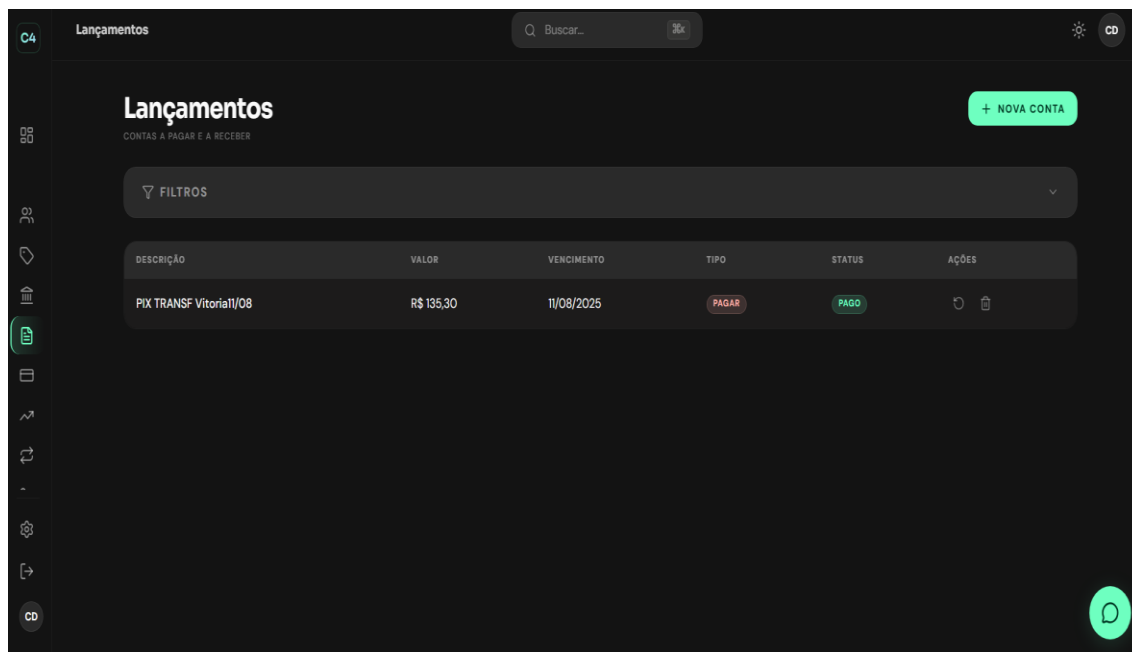
Figura 9 - Página de Contas Correntes



Fonte: Elaborada pelo autor

A Figura 10 apresenta o módulo de contas correntes, no qual o usuário cadastra, visualiza e acompanha contas bancárias vinculadas ao sistema. Esse módulo sustenta operações como baixas, estornos e transferências, mantendo os saldos atualizados.

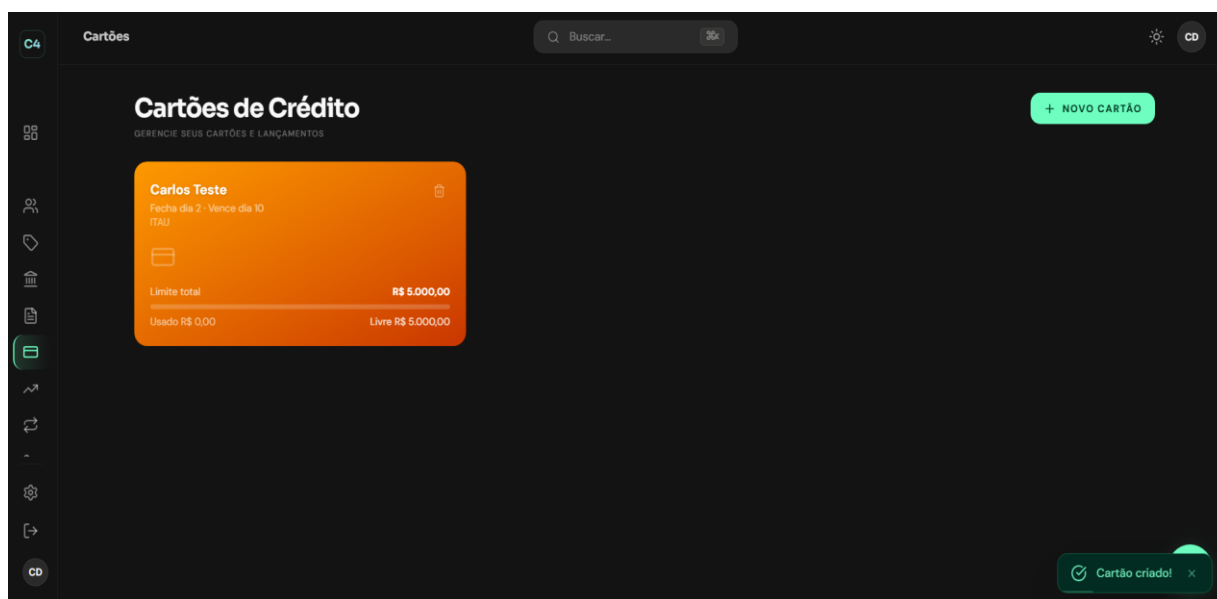
Figura 10 - Página de Lançamentos



Fonte: Elaborada pelo autor

A Figura 11 apresenta a tela de lançamentos financeiros, voltada ao registro e acompanhamento de contas a pagar, contas a receber, receitas, despesas e movimentações associadas ao fluxo de caixa do usuário.

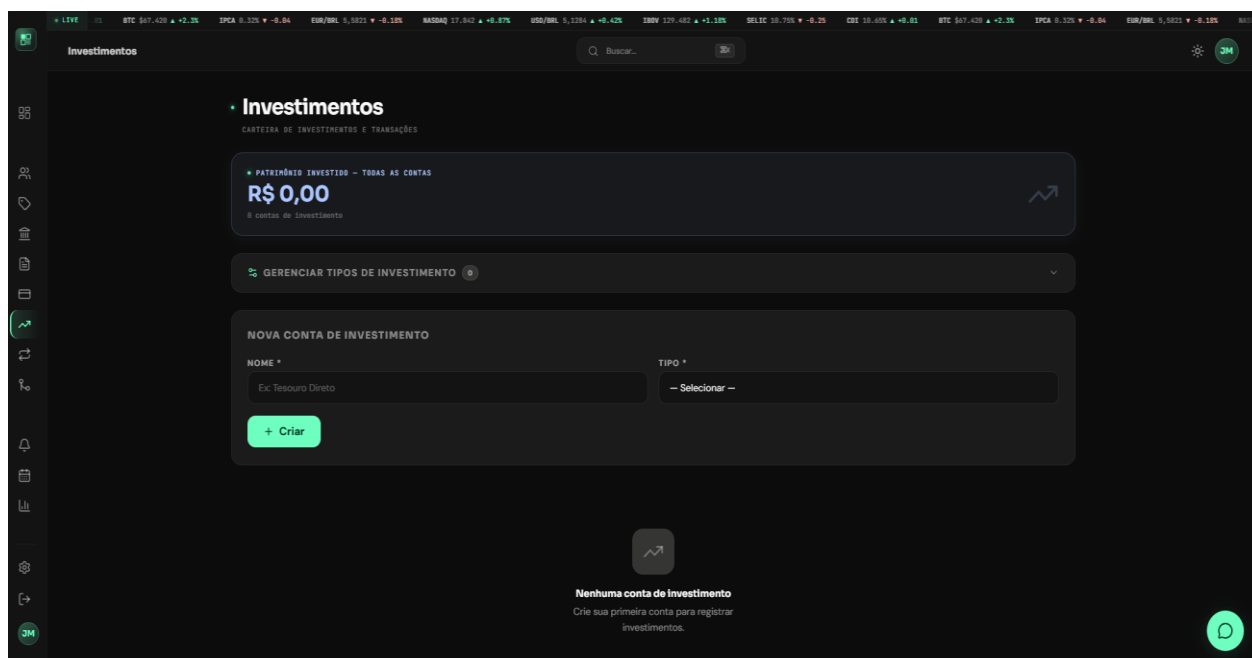
Figura 11 - Página de cartões



Fonte: Elaborada pelo autor

A Figura 12 apresenta o módulo de cartões, utilizado para controlar cartões de crédito, lançamentos, parcelamentos, faturas, limites e datas relevantes. Esse módulo integra-se às contas financeiras quando ocorre o fechamento de fatura.

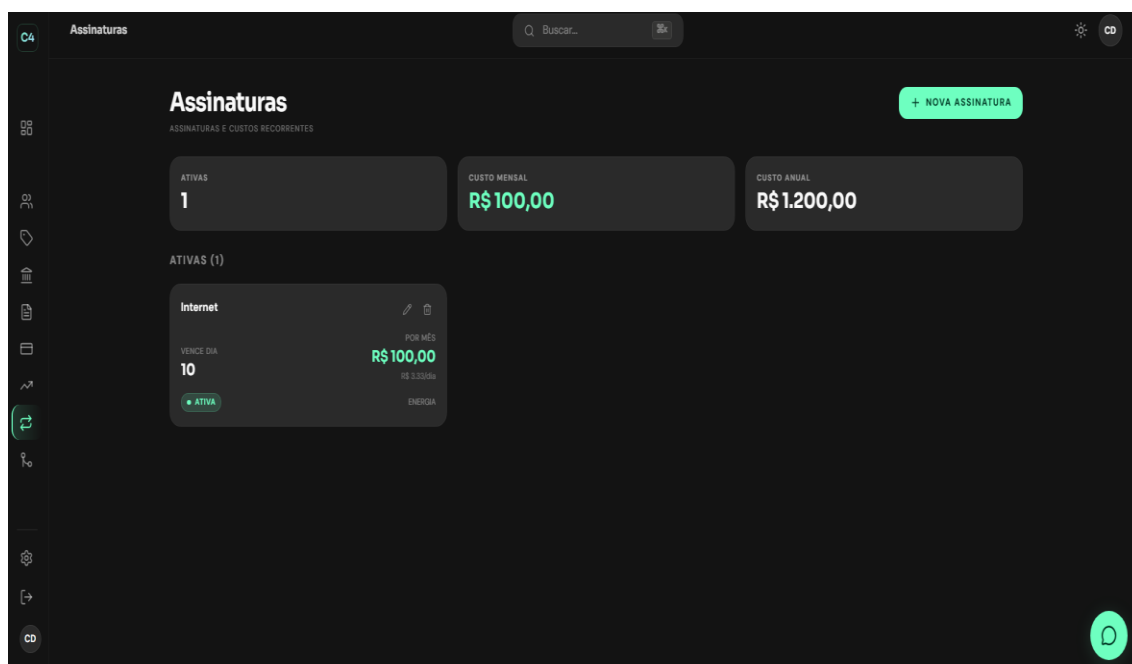
Figura 12 - Página de investimento



Fonte: Elaborada pelo autor

A Figura 13 apresenta a área de investimentos, responsável pelo acompanhamento de contas de investimento, aportes, resgates, rendimentos e composição da carteira financeira do usuário.

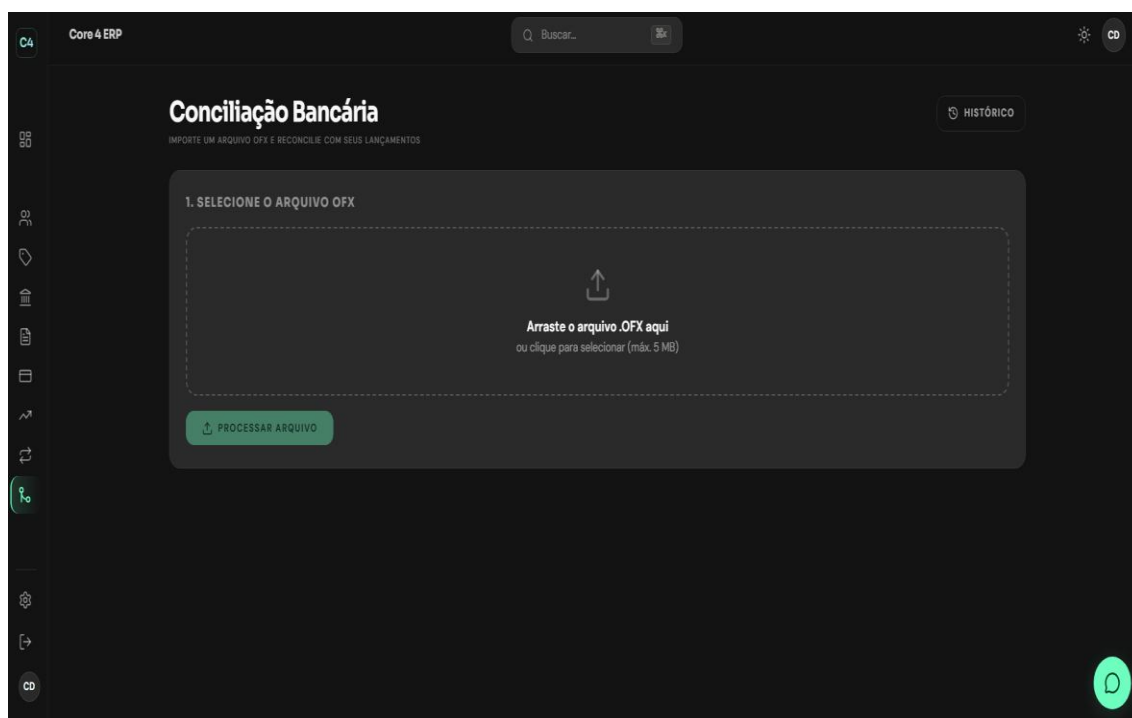
Figura 13 - Tela de Assinaturas



Fonte: Elaborada pelo autor

A Figura 14 apresenta a tela de assinaturas recorrentes, utilizada para controlar cobranças periódicas, mensalidades, serviços contratados e demais pagamentos repetitivos. Esse módulo reduz registros manuais e contribui para previsibilidade financeira.

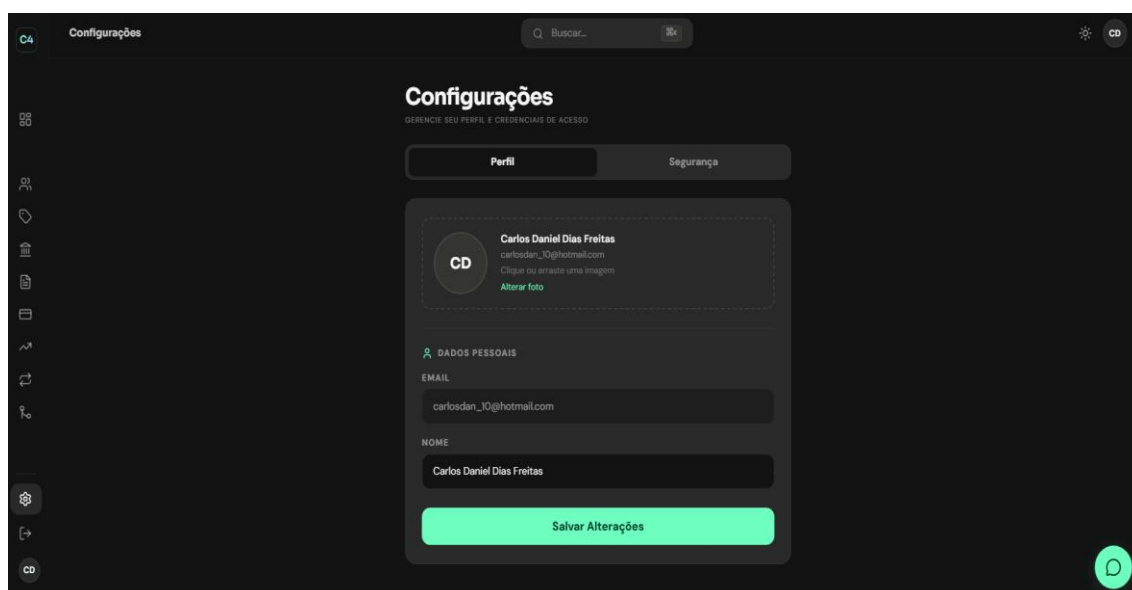
Figura 14 - Tela de Conciliação Bancária



Fonte: Elaborada pelo autor

A Figura 15 apresenta a tela de conciliação bancária, na qual o usuário importa arquivos OFX, visualiza transações bancárias e compara os itens do extrato com os registros internos do sistema. Essa funcionalidade permite vincular lançamentos, criar contas, realizar baixas, ignorar itens e finalizar a conciliação.

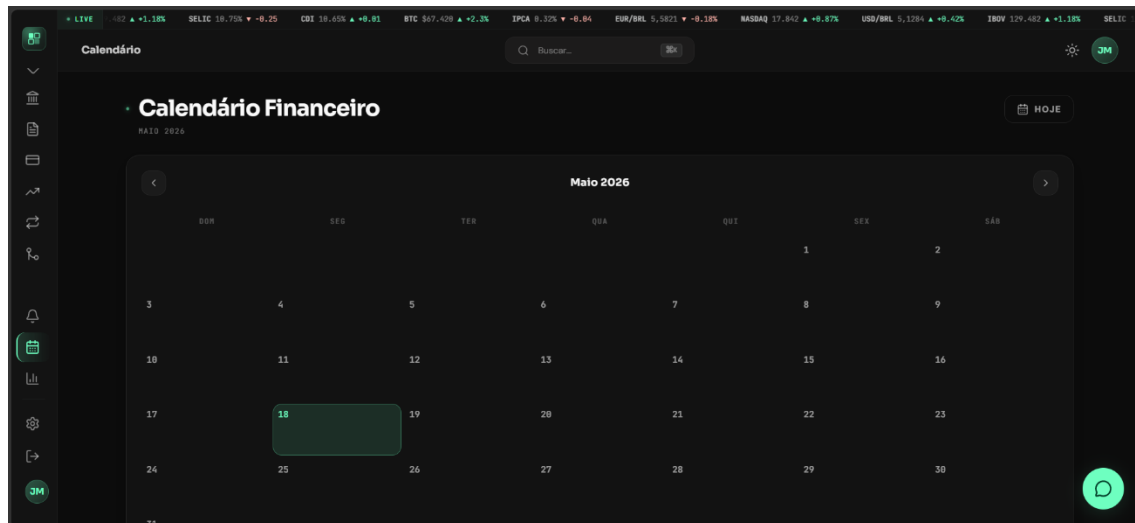
Figura 15 - Tela de Configurações da conta



Fonte: Elaborada pelo autor

A Figura 16 apresenta a tela de configurações da conta, destinada à atualização de dados do perfil, ajustes de preferências e recursos relacionados à segurança e ao gerenciamento da conta do usuário.

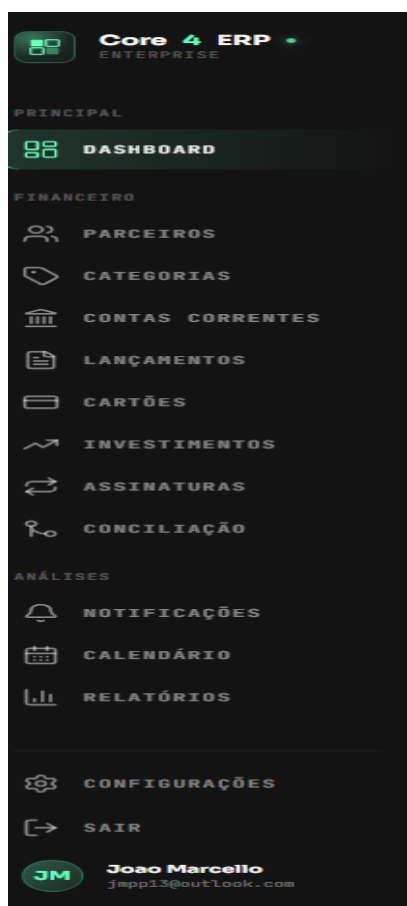
Figura 16 - Tela de Calendário financeiro



Fonte: Elaborada pelo autor

A Figura 17 apresenta o calendário financeiro mensal, que organiza contas a pagar, contas a receber, contas pagas ou recebidas e assinaturas por data de vencimento. Essa visualização facilita o acompanhamento de compromissos futuros e permite realizar baixas diretamente pela interface.

Figura 17 - Menu suspenso Lateral



Fonte: Elaborada pelo autor

A Figura 18 apresenta o menu lateral recolhível, componente de navegação principal do sistema. Esse recurso facilita o acesso aos diferentes módulos da aplicação e contribui para uma experiência de uso mais organizada.

6. MODELAGEM CONCEITUAL E FÍSICA

A modelagem conceitual e física do Core 4 ERP foi estruturada com base nas entidades essenciais ao funcionamento dos módulos financeiros, buscando garantir organização, integridade, rastreabilidade e escalabilidade das informações ao longo de todo o ciclo de vida do sistema. Cada entidade foi definida considerando as regras de negócio e os fluxos operacionais, permitindo que os dados sejam armazenados de forma consistente e preparados para suportar crescimento futuro e aumento de volume transacional.

O PostgreSQL é responsável por armazenar os dados em tabelas relacionais normalizadas, com o objetivo de eliminar redundâncias e garantir que cada informação

seja armazenada em um único ponto, facilitando a manutenção e a consistência dos registros (POSTGRESQL, 2026). Para a identificação única dos registros, são utilizadas chaves primárias numéricas do tipo *bigint*, escolha que decorre da necessidade de suportar volumes transacionais elevados sem risco de estouro de identificador, prática recomendada em sistemas financeiros com crescimento contínuo de registros.

Os relacionamentos entre as tabelas são estabelecidos por meio de chaves estrangeiras, garantindo a integridade referencial e assegurando que as associações entre as informações sejam mantidas de forma correta e confiável. O versionamento das alterações estruturais é gerenciado pelo flyway, permitindo rastrear e aplicar migrations de forma controlada ao longo do ciclo de desenvolvimento. Esse conjunto de práticas fortalece a robustez da base de dados e sustenta a confiabilidade das operações financeiras do sistema.

A escolha por identificadores numéricos do tipo *bigint* nas chaves primárias das principais tabelas foi adotada considerando o crescimento progressivo dos registros financeiros e a necessidade de manter bom desempenho em consultas, relacionamentos e índices. Segundo a documentação do PostgreSQL (2026), o tipo *bigint* utiliza 8 bytes e oferece uma faixa ampla de valores, sendo adequado para tabelas com grande volume de registros. Embora o PostgreSQL possua suporte nativo ao tipo *uuid*, armazenado em 16 bytes, sua adoção não foi priorizada neste projeto porque os identificadores do sistema não precisam ser globalmente únicos entre diferentes bancos ou ambientes distribuídos. Além disso, quando *uuids* são armazenados de forma textual, como *varchar* ou *text*, o custo de armazenamento e indexação tende a ser maior, pois o valor passa a ser tratado como cadeia de caracteres. Dessa forma, a utilização de *bigint* favorece chaves menores, índices mais compactos e relacionamentos mais simples entre tabelas fortemente relacionadas por chaves estrangeiras.

A adoção do PostgreSQL como sistema gerenciador de banco de dados justifica-se pela necessidade de integridade, consistência e suporte a operações relacionais. O PostgreSQL oferece recursos como chaves primárias, chaves estrangeiras, constraints, transações e tipos de dados adequados à modelagem de sistemas financeiros (POSTGRESQL, 2026). Esses recursos são essenciais no CORE 4 ERP porque determinadas operações não podem gerar registros parciais ou inconsistentes. A baixa de uma conta, por exemplo, envolve alteração de status, criação de registro em *tb_conta_baixada* e atualização do saldo da conta corrente. Da mesma forma,

transferências, estornos, fechamento de faturas e conciliações bancárias dependem de consistência entre diferentes tabelas.

Outro aspecto relevante da modelagem foi a normalização dos dados. A normalização busca organizar tabelas e atributos de modo a reduzir redundâncias, evitar anomalias de inserção, atualização e exclusão, e melhorar a integridade das informações. Conforme Elmasri e Navathe (2011), o projeto adequado de bancos relacionais deve considerar dependências entre atributos e decomposição de relações para reduzir duplicidades e preservar consistência. No Core 4 ERP, essa lógica aparece na separação entre entidades como *tb_usuario*, *tb_categoria*, *tb_parceiro*, *tb_conta*, *tb_conta_baixada*, *tb_conta_corrente*, *tb_cartao_credito*, *tb_fatura_cartao*, *tb_lancamento_cartao*, *tb_assinatura*, *tb_conciliacao*, *tb_conciliacao_item*, *tb_conta_investimento*, *tb_tipo_investimento*, *tb_transacao_investimento* e *tb_notificacao*.

O versionamento da estrutura do banco de dados foi realizado por meio do Flyway, ferramenta de migrations utilizada para controlar a evolução do schema ao longo do desenvolvimento. De acordo com a documentação da Redgate (2026), o Flyway permite aplicar alterações incrementais no banco por meio de scripts versionados, possibilitando rastrear a evolução da estrutura e manter diferentes ambientes sincronizados. Essa escolha foi importante porque o Core 4 ERP foi desenvolvido de forma incremental: à medida que novos módulos foram implementados, novas tabelas, campos, constraints e relacionamentos precisaram ser adicionados. Com o uso de migrations, alterações como criação de tabelas de assinatura, conciliação, bloqueio de login, tokens de redefinição de senha e regras de negócio puderam ser registradas de maneira controlada, reduzindo inconsistências entre o modelo físico e a aplicação.

Dessa forma, a modelagem física do Core 4 ERP foi construída com foco em integridade, desempenho e evolução controlada. A utilização de bigint nas chaves primárias favorece identificadores compactos e adequados a tabelas de grande volume; o PostgreSQL oferece os mecanismos relacionais necessários para manter consistência entre entidades financeiras; a normalização reduz redundâncias e melhora a manutenção dos dados; e o Flyway permite controlar a evolução do banco conforme novos módulos são adicionados.

O Modelo Entidade-Relacionamento do Core 4 ERP representa a estrutura física da base de dados, suas tabelas, chaves primárias, chaves estrangeiras e relacionamentos.

O DER demonstra a organização relacional do sistema, com entidades vinculadas ao usuário autenticado e módulos voltados à gestão bancária, contas, cartões, investimentos, conciliações e interações. Por se tratar de um diagrama extenso e de leitura detalhada, sua versão completa encontra-se disponível no Apêndice A, Figura 23.

6.1 Principais Entidades

A partir da modelagem relacional descrita no DER, é possível compreender como o domínio financeiro foi decomposto em entidades principais e relacionamentos. Esta seção sintetiza as tabelas centrais do banco de dados e sua função dentro do sistema, conectando a modelagem física aos módulos descritos anteriormente. Assim, a lista de entidades não deve ser interpretada como uma enumeração isolada, mas como a representação dos elementos que sustentam autenticação, cadastros auxiliares, contas, cartões, investimentos, conciliações, notificações e relatórios.

- *tb_usuario*: armazena dados de autenticação, perfil e papel do usuário.
- *tb_categoria*: organiza classificações de contas e lançamentos por usuário.
- *tb_parceiro*: representa clientes, fornecedores ou ambos, associados ao usuário.
- *tb_conta_corrente*: representa contas bancárias e seus saldos.
- *tb_assinatura*: representa cobranças recorrentes vinculadas a categoria, cartão, parceiro e usuário.
- *tb_conta*: representa contas a pagar ou receber, com status e vencimento.
- *tb_conta_baixada*: registra a baixa de uma conta e a conta corrente movimentada.
- *tb_cartao_credito*: representa cartões vinculados a contas correntes.
- *tb_lancamento_cartao*: representa compras e lançamentos em cartões.
- *tb_fatura_cartao*: representa faturas mensais e sua associação com conta a pagar quando fechada.
- *tb_conciliacao*: registra processos de conciliação bancária importados de OFX para uma conta corrente.
- *tb_conciliacao_item*: registra cada transação OFX conciliada, vinculada ou não a uma conta/baixa.
- *tb_conta_investimento*: representa carteiras ou contas de investimento.
- *tb_tipo_investimento*: armazena tipos personalizados de investimento cadastrados por usuário.
- *tb_transacao_investimento*: registra aportes, resgates e rendimentos.

- *tb_notificacao*: registra notificações de vencimento e fatura para o usuário.

7. CONSIDERAÇÕES FINAIS

O desenvolvimento do Core 4 ERP permitiu aplicar conceitos fundamentais de Engenharia de Software, Sistemas de Informação, modelagem de dados, segurança, usabilidade e observabilidade na construção de uma aplicação web voltada à gestão financeira pessoal. O projeto consolidou uma arquitetura composta por frontend em React, backend em Java com Spring Boot e banco de dados PostgreSQL, estruturada em camadas e organizada por módulos financeiros integrados. Dessa forma, o trabalho demonstrou que princípios tradicionalmente associados a sistemas ERP empresariais podem ser adaptados ao contexto individual, oferecendo ao usuário uma plataforma centralizada para registro, acompanhamento e análise de suas informações financeiras.

Retomando a hipótese definida no início do trabalho, considera-se que ela foi confirmada no contexto avaliado. A implementação demonstrou que uma aplicação web desenvolvida com arquitetura em camadas, autenticação segura, persistência relacional, relatórios, dashboard, conciliação bancária, calendário financeiro e assistente de IA pode centralizar registros, reduzir a dispersão das informações e apoiar a tomada de decisão do usuário. Essa confirmação é sustentada pela entrega dos módulos implementados, pela validação funcional dos principais fluxos, pela pesquisa de satisfação com usuários e pelo monitoramento técnico realizado durante a utilização do sistema.

A solução desenvolvida atende aos principais fluxos de organização financeira pessoal, contemplando autenticação e perfil, categorias, parceiros, contas a pagar e a receber, parcelamentos, baixas, estornos, contas correntes, transferências, cartões de crédito, lançamentos, faturas, assinaturas recorrentes, investimentos, notificações, calendário financeiro, dashboard, conciliação bancária por importação OFX, relatórios financeiros, exportação em Excel, geração de PDF no frontend e assistente de IA. A documentação técnica por diagramas complementou o desenvolvimento ao representar a arquitetura, a infraestrutura, as entidades, os casos de uso, os fluxos financeiros e a modelagem relacional, contribuindo para tornar mais clara a relação entre requisitos, implementação e funcionamento do sistema.

Como contribuição prática, o trabalho apresenta uma solução integrada que busca superar a fragmentação comum no controle financeiro pessoal. Diferentemente de

planilhas isoladas, anotações manuais ou ferramentas desconectadas, o Core 4 ERP reúne módulos operacionais, analíticos e de apoio à decisão em uma mesma aplicação. A integração entre contas, cartões, contas correntes, investimentos, assinaturas, relatórios, dashboard, calendário, notificações e conciliação bancária permite que o usuário acompanhe sua situação financeira de forma mais organizada e orientada por dados. Além disso, recursos como assistente de IA, monitoramento por métricas e exportação de relatórios demonstram a aplicação prática de tecnologias atuais em um problema cotidiano e socialmente relevante.

No âmbito acadêmico, este trabalho contribui ao apresentar um estudo aplicado que articula fundamentação teórica, elicitação de requisitos, modelagem, implementação e validação de um sistema de informação financeiro pessoal. A contribuição acadêmica não se limita ao produto desenvolvido, mas também ao processo de transformação de um problema real em uma solução computacional documentada, estruturada e avaliável. O trabalho demonstra como conceitos de educação financeira, sistemas ERP, arquitetura em camadas, padrão MVC, segurança da informação, modelagem relacional, usabilidade e observabilidade podem ser integrados em um projeto de software completo. Assim, o Core 4 ERP serve como exemplo acadêmico de aplicação prática de conhecimentos de Engenharia de Software em um domínio que envolve dados sensíveis, regras financeiras, integração entre módulos e necessidade de apoio à tomada de decisão.

Outra contribuição acadêmica está na documentação do processo de desenvolvimento e das decisões técnicas adotadas. A especificação de requisitos, os diagramas estruturais e comportamentais, a modelagem conceitual e física, a descrição das regras de negócio, a análise das tecnologias utilizadas e a validação com usuários formam um conjunto de evidências que permite compreender como a solução foi construída e por que determinadas escolhas foram realizadas. Essa documentação favorece a reprodutibilidade parcial do estudo, apoia trabalhos futuros relacionados a sistemas financeiros pessoais e contribui para discussões sobre adaptação de conceitos de ERP ao contexto individual.

Os resultados da validação também reforçam a relevância da proposta. A pesquisa de satisfação indicou percepção positiva dos usuários quanto à utilidade do sistema, especialmente em funcionalidades como dashboard, relatórios, segurança dos dados, conciliação bancária, calendário financeiro e assistente de IA. O monitoramento técnico, por sua vez, permitiu observar o comportamento operacional da aplicação durante o uso,

complementando a avaliação subjetiva dos participantes com evidências de desempenho, disponibilidade e estabilidade. Essa combinação entre percepção de uso e métricas técnicas fortalece a análise do sistema e demonstra a importância de avaliar soluções computacionais sob diferentes perspectivas.

Apesar dos resultados obtidos, o trabalho apresenta limitações. A validação com usuários teve caráter exploratório e foi realizada com amostra selecionada por conveniência, não permitindo generalizações estatísticas. A cobertura de testes automatizados ainda é limitada, especialmente para regras financeiras críticas e fluxos transacionais. Também permanecem como limitações a ausência de importação CSV, a inexistência de uma trilha de auditoria imutável em *jsonb*, a geração de PDF concentrada no frontend e a necessidade de aprimoramentos em desempenho e experiência mobile, aspectos apontados na avaliação dos usuários.

Dessa forma, conclui-se que o Core 4 ERP atingiu o objetivo proposto ao entregar uma aplicação funcional, modular e integrada para gestão financeira pessoal. O sistema demonstrou capacidade de organizar informações financeiras, apoiar o acompanhamento de receitas e despesas, oferecer relatórios e recursos de análise, além de incorporar mecanismos de segurança, observabilidade e apoio inteligente à tomada de decisão. As limitações identificadas não invalidam os resultados alcançados, mas indicam possibilidades concretas de continuidade, aprofundamento e evolução técnica da solução.

7.1 Trabalhos Futuros

A partir dessas limitações e das oportunidades observadas durante a validação, torna-se possível estabelecer caminhos para a continuidade do projeto. Assim, os trabalhos futuros apresentados a seguir não representam apenas complementos opcionais, mas desdobramentos naturais da pesquisa, voltados ao aumento da segurança, da confiabilidade, da cobertura de testes, da conformidade com a LGPD, da capacidade de importação de dados, da padronização de relatórios e da maturidade operacional do sistema.

- Implementar trilhas de auditoria detalhadas com armazenamento de estados anteriores e novos das entidades alteradas.
- Ampliar a cobertura de testes unitários e de integração para regras financeiras críticas.

- Aprimorar recursos de conformidade com LGPD, incluindo exclusão de conta, anonimização e política de retenção.
- Melhorar a documentação operacional e criar scripts de implantação.
- Implementar a importação de arquivos CSV para conciliação e cadastro de movimentações financeiras, ampliando os formatos aceitos além do OFX.
- Implementar a geração centralizada de relatórios em PDF pelo backend, padronizando a emissão dos documentos e reduzindo a dependência da geração realizada no frontend.

REFERÊNCIAS BIBLIOGRÁFICAS

APACHE SOFTWARE FOUNDATION. **Apache POI: component overview**. 2026. Disponível em: <<https://poi.apache.org/components/index.html>>. Acesso em: 22 abr. 2026.

BANCO CENTRAL DO BRASIL. **Endividamento de risco no Brasil**. Série Cidadania Financeira: estudos sobre educação, proteção e inclusão, n. 6. Brasília: Banco Central do Brasil, 2020. Disponível em: <https://www.bcb.gov.br/content/cidadaniafinanceira/documentos_cidadania/serie_cidadania/serie_cidadania_financeira_6_endividamento_risco.pdf>. Acesso em: 9 maio 2026.

BASTOS, Danilo Melo Fernandes. **A importância dos sistemas ERP na gestão financeira das empresas**. 2024. Trabalho de Conclusão de Curso (Graduação em Administração) – Pontifícia Universidade Católica de Goiás, Goiânia, 2024. Disponível em: <<https://repositorio.pucgoias.edu.br/jspui/handle/123456789/8488>>. Acesso em: 11 maio 2026.

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018**. Lei Geral de Proteção de Dados Pessoais (LGPD). Diário Oficial da União: seção 1, Brasília, DF, 15 ago. 2018.

ELMASRI, Ramez; NAVATHE, Shamkant B. **Sistemas de banco de dados**. 6. ed. São Paulo: Pearson Addison Wesley, 2011.

GFLEC. **S&P Global Financial Literacy Survey: participating countries**. Washington, DC: Global Financial Literacy Excellence Center, 2014. Disponível em: <<https://gflec.org/wp-content/uploads/2016/02/Gallup-country-list-with-score.pdf>>. Acesso em: 16 abr. 2026.

GIL, Antonio Carlos. **Como elaborar projetos de pesquisa**. 7. ed. São Paulo: Atlas, 2022.

GRAFANA LABS. **Grafana documentation: dashboards**. 2026. Disponível em: <<https://grafana.com/docs/grafana/latest/dashboards/>>. Acesso em: 26 abr. 2026.

ISTQB. **Certified Tester Foundation Level Syllabus v4.0.1**. International Software Testing Qualifications Board, 2024. Disponível em: <https://istqb.org/wp-content/uploads/2024/11/ISTQB_CTFL_Syllabus_v4.0.1.pdf>. Acesso em: 18 maio 2026.

JUNIT. **JUnit 5 User Guide**. 2026. Disponível em: <<https://junit.org/junit5/docs/current/user-guide/>>. Acesso em: 7 maio 2026.

LUSARDI, Annamaria; MITCHELL, Olivia S. **The economic importance of financial literacy: theory and evidence**. Journal of Economic Literature, Nashville, v. 52, n. 1, p. 5-44, 2014. Disponível em: <<https://www.aeaweb.org/articles?id=10.1257/jel.52.1.5>>. Acesso em: 28 abr. 2026.

MARION, José Carlos. **Contabilidade básica**. 12. ed. São Paulo: Atlas, 2018.

MICROSOFT. **Dynamics 365 Finance**. 2026. Disponível em: <<https://www.microsoft.com/en-us/dynamics-365/products/finance>>. Acesso em: 3 maio 2026.

NIELSEN, Jakob. **Usability engineering**. San Francisco: Morgan Kaufmann, 1994.

OECD. **Improving financial literacy: analysis of issues and policies**. Paris: OECD Publishing, 2005. Disponível em: <https://www.oecd.org/content/dam/oecd/en/publications/reports/2005/11/improving-financial-literacy_g1gh5cd2/9789264012578-en.pdf>. Acesso em: 19 abr. 2026.

OECD. **OECD/INFE 2023 international survey of adult financial literacy**. OECD Business and Finance Policy Papers, n. 39. Paris: OECD Publishing, 2023. Disponível em: <<https://doi.org/10.1787/56003a32-en>>. Acesso em: 24 abr. 2026.

OECD. **Financial education**. 2026. Disponível em: <<https://www.oecd.org/en/topics/sub-issues/financial-education.html>>. Acesso em: 12 maio 2026.

ORACLE NETSUITE. **NetSuite Cloud ERP**. 2026. Disponível em: <<https://oracle.netsuite.com/>>. Acesso em: 6 maio 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL documentation**. 2026. Disponível em: <<https://www.postgresql.org/docs/>>. Acesso em: 15 maio 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL documentation: constraints**. 2026. Disponível em: <<https://www.postgresql.org/docs/current/ddl-constraints.html>>. Acesso em: 21 maio 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL documentation: numeric types**. 2026. Disponível em: <<https://www.postgresql.org/docs/current/datatype-numeric.html>>. Acesso em: 5 maio 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL documentation: UUID type**. 2026. Disponível em: <<https://www.postgresql.org/docs/current/datatype-uuid.html>>. Acesso em: 14 maio 2026.

PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de software: uma abordagem profissional**. 8. ed. Porto Alegre: AMGH, 2016.

PROMETHEUS AUTHORS. **Prometheus documentation: overview**. 2026. Disponível em: <<https://prometheus.io/docs/introduction/overview/>>. Acesso em: 27 abr. 2026.

REACT. **React documentation**. 2026. Disponível em: <<https://react.dev/learn>>. Acesso em: 20 maio 2026.

REDGATE. **Flyway documentation**. 2026. Disponível em: <<https://documentation.red-gate.com/flyway>>. Acesso em: 8 maio 2026.

SAP. **What is SAP ERP?** 2026. Disponível em: <<https://www.sap.com/products/erp/what-is-sap-erp.html>>. Acesso em: 29 abr. 2026.

SIERRA, Kathy; BATES, Bert; GEE, Trisha. **Head First Java**. 3. ed. Sebastopol: O'Reilly Media, 2022.

SOMMERVILLE, Ian. **Engenharia de software**. 9. ed. São Paulo: Pearson Prentice Hall, 2011.

SPRING. **Spring Boot Reference Documentation**. 2026. Disponível em: <<https://docs.spring.io/spring-boot/reference/>>. Acesso em: 23 abr. 2026.

SPRING. **Spring Security Reference**. 2026. Disponível em: <<https://docs.spring.io/spring-security/reference/>>. Acesso em: 4 maio 2026.

SPRING BOOT. **Testing**. 2026. Disponível em: <<https://docs.spring.io/spring-boot/reference/testing/>>. Acesso em: 17 maio 2026.

STALLINGS, William. **Cryptography and network security: principles and practice**. 6. ed. Boston: Pearson, 2015.

TOTVS. **TOTVS Backoffice - Linha Protheus**. 2026. Disponível em: <<https://www.totvs.com/sistema-de-gestao/totvs-backoffice-linha-protheus/>>. Acesso em: 13 maio 2026.

URMA, Raoul-Gabriel; WARBURTON, Richard. **Real-world software development: a project-driven guide to fundamentals in Java**. Sebastopol: O'Reilly Media, 2020.

WIEGERS, Karl; BEATTY, Joy. **Software requirements**. 3. ed. Redmond: Microsoft Press, 2013.

APÊNDICE A – DIAGRAMAS DO SISTEMA

Em razão das dimensões dos diagramas produzidos durante o desenvolvimento do sistema Core 4 ERP, as figuras listadas a seguir foram disponibilizadas em repositório externo no Google Drive, garantindo sua visualização em resolução adequada. O acesso pode ser realizado mediante os links indicados abaixo.

Figura 18 – Diagrama de Arquitetura do Sistema

Disponível em: https://drive.google.com/drive/folders/10meWkWiN7dBfcHLW3e1OoQjqSGwmY8F8?usp=drive_link. Acesso em: maio 2026.

Figura 19 – Diagrama de Infraestrutura

Disponível em: https://drive.google.com/drive/folders/1gUtUKtJa9PAUAE7HkQAwoAtMELpPhBtI?usp=drive_link. Acesso em: maio 2026.

Figura 20 – Diagrama de Classes (Entidades Java)

Disponível em: https://drive.google.com/drive/folders/1dm6MTh_WvpyXWU-I1Bi5g6gK-SnM8lwv?usp=drive_link. Acesso em: maio 2026.

Figura 21 – Diagrama de Sequência (Fluxo de Baixa/Conciliação de Conta)

Disponível em: https://drive.google.com/drive/folders/1PZhSSpfvDZpc96AvKDT1jo8SeBoXVpHe?usp=drive_link. Acesso em: maio 2026.

Figura 22 – Diagrama de Casos de Uso

Disponível em: https://drive.google.com/drive/folders/1tEqiznFatLl_2jjHBobRYlJPGViPqXNz?usp=drive_link. Acesso em: maio 2026.

Figura 23 – Modelo Entidade-Relacionamento (DER)

Disponível em: https://drive.google.com/drive/folders/1T5da82G6Idyxe6m4IKbR8aMMDsadvyvn?usp=drive_link. Acesso em: maio 2026.